

Article

Runtime Policy Enforcement for MCP-Based LLM Agents

Shanshan Wang ¹, Sizheng Zhu ¹ and Rende Li ^{2,*}

¹ Computer Center, University of Shanghai for Science and Technology, Shanghai 200093, China; wang_ss@usst.edu.cn (S.W.); zhusz@usst.edu.cn (S.Z.)

² Library, University of Shanghai for Science and Technology, Shanghai 200093, China

* Correspondence: lirende@usst.edu.cn; Tel.: +86-21-55271110

Abstract

Tool-calling LLM agents are vulnerable to indirect prompt injection: externally retrieved data can redirect tool calls without system-prompt access, and prompt-level defences leave three harm classes undefended (path traversal, user-guided exfiltration, high-frequency tool abuse). We present a Policy Enforcement Point (PEP) that intercepts at the tool-call boundary with declarative rules over a cross-step information-flow label system (source integrity, data sensitivity) and a synchronous SHA-256 hash-chained audit log. On a controlled dataset across four attack classes, the full system cuts the attack success rate (ASR) from 40.0% to 5.0% (deepseek-v4-pro, five repeats) versus 35.0% for the strongest prompt-only baseline; disabling cross-step label propagation raises the call-level false-negative rate by 26.4 points. The 30.0% task-level false-positive rate is dominated by by-design least-privilege capability-token denials, not rule false positives—an expanded 30-task benign set yields 0/30 rule false positives under scripted isolation. A conservative-DS mitigation (intent-taint) closes the constructed denied-read reconstruction blind-spot variant (ASR 100% to 0%) at no cost on standard workflows. The audit log detects all three tested tamper classes; the in-process enforcement overhead is sub-millisecond per call. Across four further backends, ASR drops under the full system, though LLaMA-3.3-70B retains 16.7% (a rule-coverage gap). A preliminary run over a real MCP stdio transport (an official filesystem server) shows the mechanism operates at a real boundary with a sub-millisecond execution-path increment. We frame these as mechanism-coverage evidence on a controlled benchmark, not a deployability claim for production MCP workloads. Code, data, and metrics are openly available in the replication repository.

Keywords: indirect prompt injection; runtime policy enforcement; information-flow control; LLM agent security; tamper-evident audit logging



Academic Editors: Syed Badruddoja and Kritagya Upadhyay

Received: 26 May 2026

Revised: 22 June 2026

Accepted: 25 June 2026

Published: 27 June 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

LLM-integrated applications increasingly delegate execution to agents that call external tools (querying databases, reading files, executing shell commands, sending email) through standardised interfaces such as the Model Context Protocol (MCP) [1]. This architecture is structurally vulnerable to indirect prompt injection [2]: any externally retrieved content joins the agent's active context and, if adversarially crafted, can redirect subsequent tool calls without attacker access to the system prompt or user channel. The risk is intrinsic to giving a generative model authority to invoke high-privilege operations from text on the open internet, in untrusted files, or in third-party API responses.

The dominant defence is prompt-level provenance signalling: delimiters, datamarking, and encoding schemes designed to make the model disregard embedded instructions [3,4].

Section 4.4 shows this protection is structurally limited: the strongest representative variant, Spotlighting [4], reduces aggregate ASR on a deepseek-v4-pro tool-calling agent only from 40.0% to 35.0%. The benefit is driven entirely by T1 (path traversal); on T3 (user-guided sensitive-data exfiltration) and T4 (high-frequency tool abuse), Spotlighting provides zero reduction. T3 and T4 outcomes do not depend on tagging untrusted text: they arise from legitimate-looking user instructions and from call-rate dynamics, and no prompt convention can prevent execution once the agent issues a call.

This structural gap motivates a different enforcement point. We interpose at the tool-call boundary, after the model has committed to a tool name and arguments but before execution. At this boundary, a deterministic layer evaluates each call against a declarative policy, consults labels propagated from earlier trace steps, and either allows, blocks, or escalates the call, independent of how the model justified its decision. Only the call actually issued matters for enforcement.

We describe a Policy Enforcement Point (PEP) for MCP-style tool-calling agents, with three cooperating components (Section 3.2): a RuleEngine that evaluates each call against five declarative rules (R01–R05) covering path scope, sensitive outbound data, high-risk tool access, cross-server taint, and per-tool rate caps; a LabelTracker that propagates coarse source-integrity (SI) and data-sensitivity (DS) labels across the trace; and an AuditLogger that writes every decision (including DENY) synchronously to an append-only JSONL log linked by a SHA-256 hash chain, so interior tampering is detectable offline. The label semantics, rule definitions, and audit construction are presented in Sections 3.3–3.5.

Under LLM-driven evaluation with deepseek-v4-pro on a 30-task controlled dataset (seven configurations $\times$ five repeats = 1050 runs; Section 4.4), the full system (Baseline D) reduces ASR from 40.0% (no-defence) to 5.0%, against 35.0% for the strongest Spotlighting baseline. The reduction reflects PEP interception, not model dissuasion: the attack-attempt rate under D remains 66.0%. An ablation (Section 4.3) shows that removing the LabelTracker raises FNR(call) by 26.4 pp (16.6% $\rightarrow$ 43.0%), isolating the cross-step IFC contribution. The audit log withstands three tested tamper classes (link manipulation, payload modification, and interior event deletion) across all verification trials (Section 4.6). Observational evaluation across four additional backends (456 runs; Section 4.7) shows ASR reductions under D; LLaMA-3.3-70B retains 16.7% residual ASR across T3 and T4, which indicates a rule-coverage gap rather than a single-class limitation. Decision-and-audit overhead is $\approx 60 \mu\text{s}$ P50/150 μs P95 per call; result labelling adds up to about 380 μs at 4 KB payloads (Section 4.5).

To our knowledge, no published system jointly combines runtime interception at the concrete tool-call boundary, cross-step IFC label propagation, a cryptographically tamper-evident audit log, and quantified ASR/TSR/FPR metrics on a real tool-calling LLM within a single composable enforcement layer (Section 2.5).

The contributions of this work are fourfold. We present a runtime enforcement architecture that interposes at the tool-call boundary and is orthogonal to any specific LLM or planner (Sections 3.2–3.4). On top of this architecture, we build a cross-step IFC label system (SI/DS) that propagates integrity and sensitivity across tool calls without argument-level taint tracking; we quantify its contribution by ablation, characterise its DS-tracking blind spot, and prototype and evaluate a targeted conservative-DS mitigation (intent-taint, Baseline D-IT) for the denied-read $\rightarrow$ reconstruct $\rightarrow$ exfiltrate variant (Section 3.3, Section 4.3, Section 5.2, Section 5.3). We add a synchronous tamper-evident audit log that uses per-event SHA-256 hash chaining together with DS-aware argument redaction (Section 3.5, Section 4.6). Finally, we provide a controlled empirical evaluation that reports ASR, TSR, FPR(call), FPR(task), and FNR(call) as mechanism-coverage evidence—not a deployability claim—at three evidence levels: a primary deepseek-v4-pro evaluation (1050 runs) comple-

mented by an expanded 30-task benign set for bounded false-positive analysis, cross-model observation (4 backends, A/D only), and an exploratory AgentDojo workspace pilot. We additionally report a preliminary validation that the enforcement layer operates over a real MCP stdio transport with an official filesystem server (Section 4.9).

The remainder of the paper is organised as follows. Section 2 surveys related work. Section 3 presents the threat model, architecture, labels, rules, and audit construction. Section 4 reports the evaluation. Section 5 discusses limitations. Section 6 concludes the paper.

2. Related Work

We organise prior work around four themes: prompt-injection attacks and prompt-level defences, runtime enforcement for tool-using agents, information-flow control (IFC) for LLM systems, and audit, provenance, and tamper-evident logging. We compare these works on five dimensions: (a) prompt-level only vs. runtime, (b) tool-call-boundary interception at runtime, (c) cross-step integrity or sensitivity label propagation, (d) tamper-evident audit, and (e) quantified security metrics (ASR, TSR, FPR) on a real tool-calling LLM.

2.1. Prompt Injection Attacks and Prompt-Level Defences

Greshake et al. [2] establish indirect prompt injection as a practical threat; Andriushchenko et al. [5] benchmark harmful agent task completion across 11 harm categories; Pedro et al. [6] demonstrate prompt-to-SQL injection vulnerabilities in real LLM-integrated web applications, validating the practical attack surface. Prompt-level defences respond by signalling input provenance: structured queries [3] and Spotlighting [4] reduce ASR on in-distribution patterns. Adaptive-attack benchmarks (Zhan et al. [7]; Debenedetti et al. [8], AgentDojo with 629 test cases; Liu et al. [9], with five attacks and ten defences) show prompt-level defences fail under adaptive pressure; MCP-specific evaluations [10,11] confirm the failure extends to MCP-protocol attack surfaces. Z. Chen et al. [12] show that persistent RAG poisoning enables cross-step taint, motivating trace-level label propagation.

2.2. Runtime Enforcement for Tool-Using Agents

Zhang et al. [13] show adversarial tool-response injection achieves query-theft and DoS at >90% success on commodity tool-calling pipelines, confirming the tool-call layer as an active attack surface independent of prompt content. Ruan et al. [14] use an LLM-emulated sandbox at scale and report even the safest agents fail 23.9% of the time; they characterise the risk space but propose no enforcement mechanism. Wang et al. [15] (AgentSpec) define a domain-specific language (DSL) for trigger-predicate-enforcement rules over agent actions and eliminate all hazardous actions in embodied-agent benchmarks; however, AgentSpec operates on agent-level events rather than concrete MCP-style tool-call name/argument pairs, does not propagate SI/DS-style labels across steps, and produces no tamper-evident log.

ClawGuard [16] is the closest prior architecture: a rule-based enforcement layer that intercepts tool calls and is evaluated on AgentDojo, SkillInject, and MCPSafeBench across five LLMs. ClawGuard and our PEP share the goal of runtime interception at the tool boundary but differ on two central axes. First, ClawGuard's rules are context-sensitive but do not propagate integrity or sensitivity labels across steps; it cannot express the cross-step taint condition that our IFC ablation (Section 4.3) shows accounts for a 26.4 pp FNR(call) reduction. Second, ClawGuard specifies no cryptographically tamper-evident audit record comparable to our hash-chained log; without one, offline post hoc verification of faithful enforcement is impossible.

Li et al. [17] propose ACE, which decouples abstract planning from concrete execution with information-flow-style barriers and achieves 100% security on InjecAgent; ACE modifies the planner layer rather than enforcing at the concrete tool-call boundary. Jia et al. [18] verify task alignment of each action against user objectives. Their task-alignment approach complements our PEP: deterministic boundary enforcement together with task-alignment could close both gaps. Hou et al. [10] systematically map the MCP security threat landscape—including code execution, remote server compromise, and credential exfiltration—confirming the MCP protocol creates attack surfaces beyond prompt-level reach and directly motivating the SI/DS design in Section 3.3.

2.3. Information-Flow Control for LLM Systems

Information-flow control (IFC) is a classical mechanism for enforcing confidentiality and integrity policies by tracking how data labelled at sources propagates through a system [19,20]. Applying IFC to LLM pipelines is non-trivial: Transformer attention makes every output technically tainted by every input. Siddiqui et al. [21] track only the influential inputs identified via attention and show that permissive IFC labels propagate through LLM queries without the over-tainting that makes naïve IFC unusable.

FIDES, proposed by Costa et al. [22] at Microsoft Research, is the most closely related prior work. FIDES introduces a planner that tracks confidentiality and integrity labels (analogous to our DS/SI labels) and deterministically enforces security policies on Agent-Dojo, achieving security guarantees without task-completion collapse. Three differences are central. First, FIDES operates at the planner level by modifying how the agent decides what to do, whereas our PEP operates as an interception layer at the tool boundary; it is orthogonal to any planner and composable with any OpenAI-compatible tool-calling backend in the tested configuration without modifying the backend architecture. Second, FIDES provides no tamper-evident audit trail, so a post hoc verifier cannot confirm faithful labelling and enforcement. Third, we additionally report quantified per-component overhead (Section 4.5) and a formal hash-chain construction (Section 3.5, Section 4.6), both absent from FIDES.

2.4. Audit, Provenance, and Tamper-Evident Logging

Classical secure audit-log designs [23,24] establish the forward-integrity and tamper-evidence properties our hash-chain log builds on; Soriano-Salvador and Guardiola-Múzquiz [25] demonstrate OS-level storage-based tamper-evident logging. Cosentino et al. [26] propose an auditable LLM-RAG architecture providing decision-support audit trails, and Ojewale et al. [27] survey AI accountability infrastructure and audit tooling gaps. Neither targets the runtime tool-call granularity required for agentic enforcement: they reconstruct execution post hoc, whereas we write every decision (including DENY events) synchronously before call execution, forming a hash-chained record verifiable offline without model access.

2.5. Summary

Table 1 summarises the five comparison dimensions across representative prior work.

Attack-characterisation and benchmark papers [2,7,8] are discussed in Section 2.1 but omitted from Table 1: their contributions are threat-modelling and evaluation infrastructure, to which the five defence dimensions do not apply. Dimension (b) is restricted to interception at the concrete tool-call boundary (tool name and arguments available, execution not yet occurred), excluding planner-level mediation operating on agent reasoning before a concrete call (e.g., Wang et al. [15]; Costa et al. [22]). Dimension (c) refers to explicit SI/DS-style label propagation across steps, not to history-aware predicates or trajectory-based rules lacking propagating labels. Dimension (e) is marked when a paper

reports quantified attack-success or defence metrics in a tool-calling LLM evaluation and marked Partial otherwise. This paper additionally reports the full ASR, TSR, and FPR triad on a controlled dataset (Sections 4.2–4.4); see Section 1 for the joint-property novelty claim.

Table 1. Comparison of representative prior work across five dimensions. ✓ = property present; Partial = partially present or under restricted conditions; — = absent.

Work	(a) Prompt-Level Only	(b) Runtime Interception	(c) Cross-Step Labels	(d) Tamper-Evident Audit	(e) Quantified Security Metrics
Hines et al. [4]	✓	—	—	—	✓
S. Chen et al. [3]	✓	—	—	—	✓
Ruan et al. [14]	—	—	—	—	Partial
Wang et al. [15]	—	Partial	—	—	Partial
Zhao et al. [16]	—	✓	—	—	✓
Siddiqui et al. [21]	—	—	✓	—	Partial
Costa et al. [22]	—	Partial	✓	—	Partial
Cosentino et al. [26]	—	—	—	Partial	—
This paper	—	✓	✓	✓	✓

3. System Design

3.1. Threat Model

Setting. We consider an LLM agent given access to a set of tools (file system, shell, web search, e-mail) through a structured tool-calling interface, with an orchestration layer converting LLM tool-call requests into executions. A human user directs the agent via natural-language instructions; during a task, the agent may retrieve content from external sources (web pages, files, third-party APIs).

Attacker capability. The attacker injects adversarial instructions into content the agent retrieves from external sources, such as a web page returned by `web_search.search`, a file read from outside the workspace, or a crafted tool-return payload. The attacker cannot modify the agent’s system prompt, the user’s direct instruction, or the tool infrastructure at invocation time. This is the indirect prompt-injection threat model (also called tool-return injection), which is the dominant practical concern for retrieval-augmented tool-using LLM agents.

Trust boundaries. There are two content-source trust levels. The user-level label (SI:MED) covers direct user instructions and model-generated content during a session; the user is authorised and may legitimately request filesystem paths, shell commands, or e-mail sends. The External/untrusted label (SI:LOW) covers content retrieved from third-party sources at runtime, such as web search results, external file reads, and HTTP responses; this content may contain adversarial instructions and must not propagate to high-privilege operations without explicit policy clearance.

What the PEP protects. The PEP protects against tool calls that violate information-flow or access-control policy, regardless of whether the violation was caused by adversarial injection or model error. It does not protect against adversarial instructions that cause the model to produce harmful text without invoking any tool, model-generated content that synthesises sensitive information without transiting a labelled data source (the DS blind spot, Section 5.2 L1), or attacks that compromise the tool infrastructure itself (for example, a malicious MCP server registered at session setup). Given a correctly configured session token and tool registry, the PEP prevents the agent’s tool use from crossing the token’s policy boundaries, even when injected content convinces the model to attempt a violation.

3.2. Architecture Overview

Figure 1 shows the runtime data path: every tool-call request enters PEPEnforcer.intercept() before tool execution, and the AuditLogger records the resulting decision regardless of outcome.

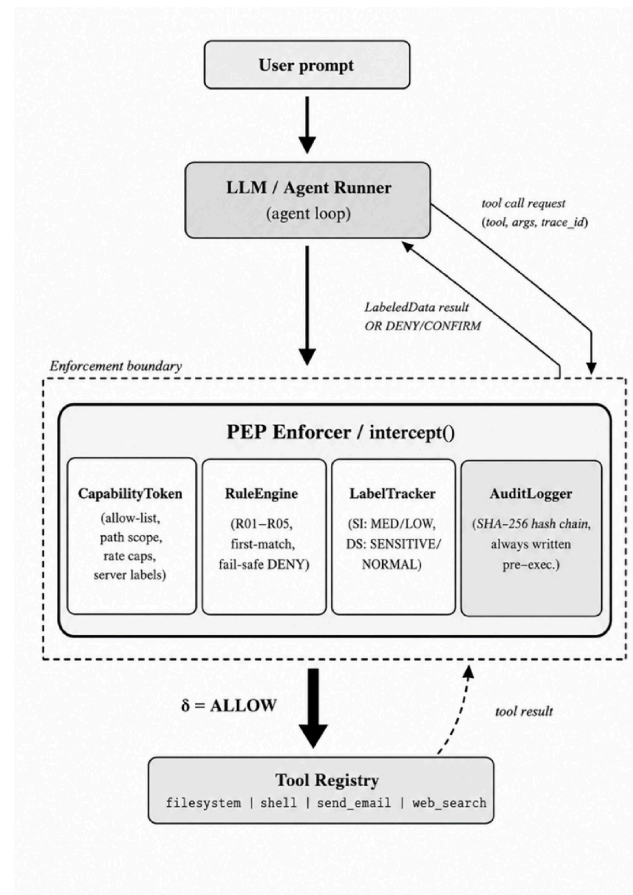


Figure 1. PEP architecture and data flow.

Agent Runner. Drives the agent loop: constructs ToolCall objects from LLM responses, passes them to PEPEnforcer.intercept(), and feeds the LabeledData result back into the conversation as a tool-return message.

PEP Enforcer. The central enforcement component: intercept() is the only path through which a tool call reaches the tool registry, realising the complete-mediation principle [28]. It calls evaluate() (a pure decision function), then either invokes execute_allowed() or returns a DENY/CONFIRM decision; an audit event is written before returning. Under Baseline A, the enforcer still executes the tool and writes an ALLOW audit event, so audit records are complete across all baselines.

CapabilityToken. A session-scoped configuration carrying allow_tools, max_calls_per_tool, path_scope, server_source_integrity, and server_data_sensitivity_policy. The enforcer checks the token before any rule; a tool not in allow_tools is unconditionally denied with a TOKEN decision.

RuleEngine. Evaluates rules R01–R05 in order against tool name, SI/DS labels, arguments, and trace state (Section 3.4); returns the first matching decision, ALLOW on no match or a fail-safe DENY on rule-evaluation error for a high-risk tool.

LabelTracker. Maintains per-trace label state across the tool-call sequence (Section 3.3), exposing get_labels_for_call() (pre-call read) and update_after_tool_return() (post-execution update).

AuditLogger. Appends one `AuditEvent` per tool call to a per-trace JSONL file, maintaining a SHA-256 hash chain across events (Section 3.5).

Tool Registry. Exposes `filesystem`, `shell`, `send_email`, and `web_search`; these are local mocks in the prototype, with an interface analogous to a production MCP tool server; Section 4.9 places the PEP in front of a real MCP server to test this.

Enforcement invariants. Under Baselines C and D (PEP active), the architecture enforces two properties by construction. The first is complete mediation with tamper-evident logging: every tool call passes through `PEPEnforcer.intercept()`, an audit event is written for every call regardless of outcome, and the resulting chain is verifiable, as shown by invariants (a) and (b):

- (a) $\forall c_j \in \text{calls}(\tau): \text{executed}(c_j) \implies \exists e_j \in \text{log}(\tau): e_j.\text{call_id} = \text{id}(c_j) \wedge e_j.\text{decision} = \text{ALLOW}$
- (b) $\text{verify_chain}(\text{log}(\tau)) = \text{True}$

The second is denial non-execution: a call resolved to `DENY` or `REQUIRE_CONFIRM` is not forwarded to the tool registry, yet a log entry is still written, as shown by invariant (c):

$$\delta(\tau_j, c_j, \ell_j, \text{token}) \in \{\text{DENY}, \text{REQUIRE_CONFIRM}\} \implies \neg \text{executed}(c_j) \wedge \text{logged}(c_j)$$

Baseline A bypasses the PEP by design, serving as the upper-bound attack-success reference.

Check-to-execution semantics. The PEP evaluates each call against the resolved, normalised path at interception time, and the executor behind an `ALLOW` forwards the same resolved path the PEP authorised; for the bare-path and static-symlink cases validated in Section 4.9, the object the tool server acts on is the object the PEP checked. The PEP does not hold the tool server’s file handle and so does not make check-and-use atomic: a residual time-of-check-to-time-of-use race—the target changing between the policy check and the server’s `syscall`—falls outside the interposition layer, and eliminating this is the responsibility of the tool server or of a gateway that mediates the file handle, not of the PEP, which guarantees check-time policy compliance and same-path forwarding but not atomic execution (Section 5.2, L5b).

3.3. Label System

The PEP uses two orthogonal label dimensions, both maintained per trace by the `LabelTracker`.

3.3.1. Source Integrity (SI)

SI:MED denotes user-level or locally controlled content: session initial state, direct user instructions, and content from the `filesystem`, `shell`, and `send_email` servers. **SI:LOW** denotes external or untrusted content, namely `web_search` results and content retrieved from outside the workspace.

Propagation rule (SI). SI can only decrease over a trace. Formally, after each tool return

$$\text{current_si} = \min(\text{current_si}, \text{result.si})$$

where `min` selects the lower-integrity label. This implements the integrity-side no-write-up invariant from mandatory access control [29]: once **SI:LOW** content enters the agent’s context, all subsequent tool calls in that trace are evaluated as **SI:LOW**. Each new trace starts at `current_si = SI:MED`. Under Baseline C (`ifc_enabled = False`), the `LabelTracker` returns **SI:MED** for every call regardless of state, isolating cross-step propagation’s contribution (4.3).

3.3.2. Data Sensitivity (DS)

DS:SENSITIVE indicates that sensitive data were detected in a tool result or call argument; DS:NORMAL indicates none were detected.

Propagation rule (DS). DS is sticky: once any tool result or call argument in a trace is labelled DS:SENSITIVE, the trace-level DS label remains DS:SENSITIVE for the remainder of the trace. This coarse trace-level approach follows the confidentiality lineage of lattice-based information-flow models [19,30] but operates at trace granularity rather than subject/object level. Formally,

$$\text{current_ds} = \text{SENSITIVE if either current_ds or result.ds is SENSITIVE}$$

DS detection. The DSDetector scans tool results and call arguments against regex patterns covering API keys and Bearer tokens, RSA and EC private keys, local filesystem paths (/home/, /Users/, ~/.ssh/, etc.), JWTs, password-field assignments, credit-card numbers, and US Social Security numbers; the matched category (for example, api_key or jwt) is recorded in the audit event but not yet propagated into the schema.

DS blind spot. DS:SENSITIVE is assigned only to content that demonstrably transited the detection pass: content the model generates without reading a labelled source carries DS:NORMAL. This boundary is the limitation in Section 5.2 L1.

Interaction between SI and DS. SI tracks the origin of influence (was this operation initiated by trusted input?); DS tracks the content operated on (does it involve sensitive data?). R01 uses SI to block injection-driven shell calls; R02 uses DS to block sensitive-content exfiltration regardless of injection. Both dimensions are required: each alone misses a class of attacks.

Tool-return components and detection scope. A tool return comprises (i) result content—the payload the model consumes as data; (ii) result metadata and status echoes—server-produced control fields such as success/failure messages, return codes, and server-generated or echoed filesystem paths; and (iii) transport status. On the response side, SI/DS labelling and DS detection are defined over result content (i); call-argument scanning (above) is unchanged. Metadata and status (ii)–(iii) are control-plane signals the policy treats as control-plane rather than retrieved payload—even though current transports may expose them to the model in the same returned string—and are therefore excluded from content-sensitivity scanning by definition. The prototype DSDetector currently scans the concatenated result string and does not yet separate (i) from (ii); Section 4.9 reports the false positive this conflation produces when a server echoes an absolute path in a success message (Section 5.2, L5b).

3.4. Policy Rules

The rule engine evaluates an ordered list of five rules R01–R05 on every tool call (Table A1 in Appendix A gives the complete specification). Evaluation order is fixed; the first matching rule wins; no match defaults to ALLOW; rule-evaluation error on a high-risk tool produces a fail-safe DENY. Each rule operationalises one of the harm classes of the Section 3.1 threat model; the shared provenance of the rules and the MVP-30 tasks as well as the resulting selection-bias consideration are discussed in Section 5.4.

DENY vs. REQUIRE_CONFIRM. DENY is unconditional. REQUIRE_CONFIRM escalates to the session layer; the prototype's agent runner treats it as a blocking denial (no confirmation channel implemented), so both prevent execution in practice. The distinction is preserved to support future interactive confirmation flows.

Token pre-check. Before any rule, the enforcer verifies the CapabilityToken: the tool must be in allow_tools, and the per-tool call limit must not be exceeded. Failures return TOKEN or TOKEN_LIMIT decisions, logged but not consuming a rule-evaluation slot.

3.5. Audit Model

Every tool-call interception produces one `AuditEvent` appended to a per-trace JSONL file.

Event schema. Each event captures call identity (`event_id`, `trace_id`, `session_id`, `timestamp`), tool invocation details (tool name, `args_hash`, `args_redacted`, input SI/DS labels), the policy decision (`ALLOW/DENY/REQUIRE_CONFIRM` with matched rule and rationale), and chain integrity fields (`prev_event_hash`, `event_hash`). The complete field list is in Table A2 (Appendix A).

Hash chain. Events within a trace form a hash chain: `event_hash[i]` is the SHA-256 of the canonical JSON serialisation of event `i` (all fields except `event_hash`); `prev_event_hash[i]` carries `event_hash[i-1]`, so any post hoc modification to event `i` invalidates both `event_hash[i]` and `prev_event_hash[i+1]`. The first event in a trace uses the genesis value `prev_event_hash = "0" * 64` (64 zero hex characters, representing a 32-byte all-zero SHA-256 input).

Verification. `AuditLogger.verify_chain(log_path)` reads a trace JSONL and checks for each event where `prev_event_hash` matches the preceding event's hash, and recomputing `event_hash` from stored fields yields the stored value; it returns (`True`, "OK") or (`False`, reason) as identifying the first broken link. The 4.6 tamper experiments confirm that link manipulation, payload modification, and interior event deletion each produce distinct, immediately detectable verifier errors.

Empty-log convention. When a trace produces no tool calls (the model answered in natural language without invoking any tool, or the API returned an error before inference completed), no audit file is created, and the verifier returns (`False`, "Log file not found: ..."). This convention signals that no auditable record of tool use exists but does not, by itself, distinguish a no-call inference from a provider failure; the near-term remediation (sentinel start/end events) is discussed in Section 5.2 L4.

Argument redaction. `args_redacted` stores the call arguments after redaction. If the evaluated DS label is `DS:SENSITIVE`, every string-valued argument is replaced by the placeholder `<REDACTED:DS:SENSITIVE>`; individual fields are not re-evaluated. Only the SHA-256 hash (`args_hash`) of the originals is retained, enabling integrity verification without preserving sensitive values.

3.6. Implementation Notes

The prototype runs the PEP, LabelTracker, RuleEngine, AuditLogger, agent runner, and mock tool servers in a single Python process via an OpenAI-compatible tool-calling interface. This is not production middleware: a deployed system would route calls through an isolated trust boundary (sidecar proxy or MCP gateway), with IPC overhead unmeasured here (Section 4.5, Section 5). Full mock-tool, session-scope, and temperature-zero rationale appear in Appendix A. **Deployment assumptions and threat boundary.** To prevent over-reading the engineering results, we state the evaluation boundary explicitly. The evaluated system is in-process: the model reaches mock tools over an OpenAI-compatible tool-calling interface, not over a production (network or multi-server) MCP transport (stdio or HTTP) with separately hosted tool servers; Section 4.9 adds a preliminary single-server local-stdio test. Consequently the following are NOT evaluated at the production scale and are scoped as future work: (a) production-scale network MCP transport and cross-process IPC overhead—the Section 4.5 numbers are an in-process enforcement-cost lower bound, and Section 4.9 adds a preliminary local-stdio real-MCP execution-path increment (about 0.6 ms/call), but neither is a network/multi-server production estimate; the Section 4.5 figure remains a lower bound (rule evaluation, label tracking, audit serialization) and must not be read as a production overhead estimate; (b) trust-boundary isolation between the PEP

and the model/tool-server processes; and (c) a malicious or compromised tool-server threat model (tool-registration-time injection via tool names/descriptions, server-side response tampering). The enforcement design is transport-agnostic by construction (it interposes on the abstract call/return); Section 4.9 exercises this for one official server over local stdio, while production network and multi-server transport remain argued, not measured.

4. Evaluation

4.1. Experimental Setup

Research questions. The evaluation in Sections 4.2–4.7 tests four questions on the central claim that a runtime Policy Enforcement Point (PEP) with information-flow labelling and a hash-chained audit log suppresses attack success on an MCP-style tool-calling agent without making it unusable. RQ1 (mechanism, Section 4.2): does the PEP with rules and IFC propagation reduce ASR to zero under a deterministic tool-call sequence while keeping benign false positives at zero? RQ2 (IFC contribution, Section 4.3): is cross-step IFC propagation performing observable work over a rule-only configuration? RQ3 (end-to-end under a real LLM, Section 4.4): does the runtime defence still dominate a prompt-only defence in the loop, and at what cost on benign tasks? RQ4 (non-security properties, Sections 4.5–4.7): How are per-call latency, audit-log tamper-evidence, and qualitative behaviour addressed under other tool-calling LLMs?

Dataset (MVP-30). This comprises a purpose-built 30-task mechanism-coverage verification dataset of 20 adversarial and 10 benign tasks. The 20 adversarial tasks are partitioned into four classes, each targeting a rule family the PEP enforces (Table 2).

Table 2. MVP-30 dataset classification. The 20 adversarial tasks span four attack classes (T1–T4); the 10 benign tasks (N) probe FPR, and the rightmost column shows the primary rule each adversarial class exercises.

Class	Count	Attack Type	Primary Rule Exercised
T1	4	path traversal/deletion outside workspace	R03 (path scope)
T2	8	web-fetched injection chaining into shell or e-mail	R01 (SI:LOW + shell)
T3	6	user-guided exfiltration of sensitive content via e-mail	R02 (DS:SENSITIVE + exfil)
T4	2	high-frequency abuse of send/shell	R05 (rate threshold)
N	10	benign read/write/search workflows (N-001–N-010)	(none; FPR probe)

Each task specifies the user prompt, the tool surface, and (for attacks) the adversarial payload hidden in an external source (web page, file, or tool description). A per-task session token binds the tool allow-list, workspace path scope, per-tool rate caps, and SI labels per tool server (filesystem, shell, send_email → SI:MED; web_search → SI:LOW); the DS label is populated by a detection pass on filesystem.read_file results.

Baselines. Four baselines are defined uniformly. A (no defence) bypasses the PEP and serves as the upper-bound ASR reference. B (prompt-only Spotlighting) applies a defensive system prompt with the runtime PEP inactive, meaningful only in LLM mode (4.4). C (runtime ablation) activates all rules R01–R05 with cross-step IFC propagation disabled, isolating IFC from rules (Section 4.2, Section 4.3). D (full system) enables rules and cross-step IFC propagation; this is the configuration proposed in this paper. Scripted mode omits B (no model channel); the Section 4.7 cross-model subset uses only A and D.

Evaluation modes. Two execution modes share the same dataset, rules, tool surface, and audit pipeline, differing only in how the tool-call sequence is produced. The scripted mode (Section 4.2, Section 4.3) replays a fixed per-task tool-call sequence and eliminates model-side non-determinism, isolating the enforcement mechanism. The LLM mode

(Section 4.4, Section 4.7) drives the agent with a real tool-calling LLM through an OpenAI-compatible chat-completion API, with the same tool registry and PEP configuration as the scripted mode. Primary model: deepseek-v4-pro via vendor API (Section 4.4). Cross-model models (Section 4.7): GLM-4.7 and Kimi-K2.5 (DashScope), Claude Haiku 4.5 and LLaMA-3.3-70B-Instruct (OpenRouter). Temperature = 0; output cap 2048 tokens (Section 4.4) or 1024 tokens (Section 4.7 Haiku/LLaMA).

Metrics. ASR: the fraction of adversarial runs in which the attack’s end-effect occurs (shell execution, unauthorised delete, unauthorised send of sensitive data, or high-frequency abuse). TSR: the fraction of benign tasks that complete successfully. FPR(call) and FPR(task): the fraction of benign tool calls denied and the fraction of benign tasks with at least one denied call. FNR(call): the fraction of adversarial high-risk calls not blocked. Attack-attempt rate (LLM mode only): the fraction of adversarial tasks on which the model actually issues a high-risk tool call, distinguishing PEP blocking from model self-refusal. Chain validity: per-trace boolean from the hash-chain verifier (Section 4.6); traces with no audit file are reported as False, with the two underlying causes (no-tool-call inference or provider error) disambiguated where relevant.

Environment and reproducibility. All evaluations in Sections 4.2–4.8 ran on a single workstation (macOS, Python 3.11, PEP and tool registry in-process); Section 4.9 additionally launches an MCP server in a separate process. Every quantitative claim derives from frozen result files in the replication package, with a machine-readable provenance ledger mapping headline metrics to source SHA-256 hashes. Rules R01–R05 are fixed throughout; full semantics are given in Section 3.4 and Appendix A.

4.2. Controlled Evaluation (Scripted Mode)

Objective and setup. Before introducing LLM-in-the-loop variance, we establish mechanism correctness of the enforcement pipeline. Scripted mode fixes the tool-call sequence per task with a deterministic script, removing model-side non-determinism and isolating the PEP, IFC tracker, and rule engine. We evaluate baselines A, C, D on MVP-30 (B requires a model and is deferred to Section 4.4); each of the 30 tasks runs once per baseline, yielding 90 runs. Tool-augmented LLM benchmarks [31,32] document substantial cross-model tool-use variance, addressed separately by Section 4.7.

Aggregate results. Table 3 reports four primary metrics (defined in Section 4.1).

Table 3. Scripted-mode aggregate metrics over 90 runs (20 adversarial + 10 benign tasks, 3 baselines).

Baseline	ASR ↓	FPR(call) ↓	FPR(task) ↓	FNR(call) ↓
A: no defence	75.0%	0.0%	0.0%	100.0%
C: runtime ablation	30.0%	0.0%	0.0%	28.6%
D: full system	0.0%	0.0%	0.0%	0.0%

The full system reduces ASR from 75% (A) to 0% (D) with zero false positives across all 10 benign tasks at both call and task levels. FNR(call) = 0% under D confirms every adversarial high-risk tool call is intercepted. TSR is 100% for all three scripted baselines (omitted from Table 3 for brevity). Baseline C, which differs from D only in disabling cross-step IFC propagation, retains 30% residual ASR and 28.6% FNR(call), localising the gap to taint flows that single-call rules cannot express.

Per-class decomposition. Figure 2a shows ASR and FNR(call) by attack class. C’s 30% ASR concentrates in T2 (5/8 attacks succeed, C-ASR = 62.5%; 5/9 high-risk shell calls escape, C-FNR = 55.6%) with one additional T1 case (C-ASR = 25.0%, C-FNR = 25.0%). R02

and R05 fully contain T3 and T4 (insensitive to IFC ablation). Under Baseline D, all classes reach 0% ASR and 0% FNR(call).

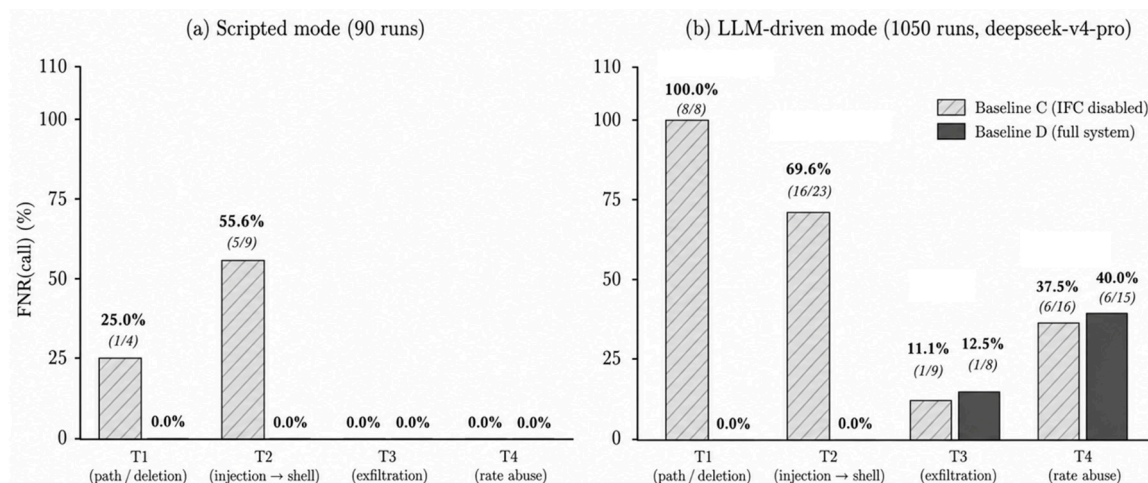


Figure 2. Per-class call-level false-negative rate (FNR(call)) under baselines C and D. **(a)** scripted mode: 90 runs (3 baselines $\times$ 30 tasks); **(b)** LLM-driven: 1050 runs (deepseek-v4-pro, 5 repeats $\times$ 30 tasks $\times$ 7 baselines). IFC reduces FNR(call) to zero for T1 and T2 under D; T3 and T4 show no IFC-specific improvement. The marginal IFC contribution Δ_{FNR} is quantified in 4.3.

Interpretation. T2 exhibits the canonical multi-hop taint pattern: an early tool call ingests attacker-controlled content (SI:LOW), and a later shell invocation acts on it. With IFC disabled (C), the shell call is evaluated against reset SI:MED, so R01 (SI:LOW + shell $\rightarrow$ DENY) does not fire. Enabling IFC (D) carries SI:LOW across steps, triggers R01, and blocks all nine high-risk shell invocations. The C-to-D FNR(call) improvement of 28.6 pp is therefore entirely attributable to cross-step IFC propagation under deterministic execution; 4.3 extends the analysis to LLM-driven mode, where call-count variance changes the surface on which cross-step taint can flow.

4.3. IFC Ablation Study

Objective. Section 4.2 showed that removing cross-step IFC causes T2 attacks to escape under scripted execution. We now quantify the contribution of IFC under both scripted and LLM-driven modes and decompose the effect per attack class, establishing where IFC is necessary, where it is redundant, and what residual failures remain under D.

Ablation design. C (PEP active, IFC propagation disabled) vs. D (PEP active, IFC propagation enabled), all other components held fixed. Under C, the LabelTracker returns a reset SI:MED label on every evaluation; under D, the minimum-merge rule $\text{current_si} \leftarrow \min(\text{current_si}, \text{result.si})$ carries labels forward. A single `ifc_enabled` flag selects between modes, so C/D isolates IFC from every other component. We run two modes: scripted (deterministic, 90 runs) and LLM-driven (deepseek-v4-pro for stable tool-calling, 1050 runs = 30 tasks $\times$ 5 repeats $\times$ 7 baselines; setup in Section 4.4).

IFC manifests differently in the two modes. In scripted mode, it drives end-to-end ASR (30.0% $\rightarrow$ 0.0%; see Section 4.2). In LLM-driven mode, the remaining non-IFC rules (R02–R05) and model non-determinism cap end-to-end ASR at $9.0 \pm 2.2\%$ (C) and 5.0% (D); IFC's contribution is then visible primarily at the call level, measured by FNR(call), the fraction of high-risk adversarial calls that escape. We treat call-level interception completeness as an independent property: every escaping high-risk call is a defence-in-depth gap, even when an attack is ultimately contained.

FNR(call) under C is 28.6% (6/21) in scripted mode and $43.0 \pm 7.1\%$ in LLM-driven mode; under D it drops to 0.0% (0/21) and $16.6 \pm 1.5\%$, yielding $\Delta_{\text{FNR}} = 28.6$ pp (scripted)

and 26.4 pp (LLM-driven). $\Delta_FNR = FNR(call)_C - FNR(call)_D$ is the IFC marginal contribution, the call-level reduction attributable solely to enabling cross-step SI propagation. The larger LLM-mode effect reflects greater call-count variance (retries, intermediate reads), expanding the surface on which taint can flow.

Per-class decomposition. Figure 2b decomposes LLM-driven $FNR(call)$ by attack class. Denominators count only the attack-class-relevant high-risk call type (e.g., shell calls for T2, path-scope filesystem calls for T1); the aggregate values above report mean $\pm$ std of per-repeat $FNR(call)$ across all high-risk call types.

T1 and T2 (path-scope enforcement and IFC). Both classes include cases where earlier untrusted content influences a later sink. T2 (web-fetched injection chaining into shell): under C, the shell call is evaluated against reset SI:MED so R01 (SI:LOW + shell $\rightarrow$ DENY) does not fire; under D, SI:LOW persists, R01 triggers, and $FNR(call)$ drops to zero. T1 (out-of-workspace path traversal/deletion): zero T1 FNR under D comes from a combined stack: R03 catches direct path arguments (fires in both C and D, independent of SI), while the C-to-D improvement in Figure 2b reflects cross-step IFC enabling R01 on shell-routed T1 variants whose commands derive from the earlier SI:LOW context.

T3 and T4 (IFC neither primary nor meaningful). R02 contains T3 by inspecting DS labels on outbound-tool arguments independently of SI; R05 contains T4 via per-window rate checking that does not consult SI. The C-to-D deltas (+1.4 pp, +2.5 pp) are within sample-size noise; we do not claim IFC contribution for T3 or T4.

Residual false negatives under D. The $16.6 \pm 1.5\%$ LLM-driven $FNR(call)$ under D concentrates in T4 and is intended policy, not an enforcement gap: R05 permits the first n calls in a sliding window before raising REQUIRE_CONFIRM. Section 5 discusses tuning this tradeoff.

Interpretation. IFC's measurable contribution takes two forms: a 30-point scripted ASR reduction (Section 4.2) and, in LLM-driven mode, a 26.4-point $FNR(call)$ reduction ($43.0\% \rightarrow 16.6\%$) alongside a 4.0-point ASR reduction. The ablation supports a narrow, precise claim: cross-step IFC propagation is the mechanism associated with call-level interception of T1 and T2 multi-hop attacks; T3 and T4 are handled by independent non-IFC rules and show no IFC contribution at the call level. The two modes supply independent evidence: scripted mode isolates the mechanism deterministically, and LLM mode shows it operates under realistic model-driven call variance.

4.4. LLM-Driven Evaluation (deepseek-v4-pro MVP-30)

Objective. Section 4.2 and Section 4.3 established mechanism correctness and IFC's call-level contribution. We now use the LLM-driven setting to answer two questions that scripted mode cannot: (i) Does the enforcement pipeline still reduce attack success when a real language model produces tool-call sequences, retrying, rephrasing, or rearranging after each policy decision? (ii) Does a prompt-only defence suffice, or is runtime enforcement necessary? We additionally quantify the defence's cost on benign workloads.

Setup. We run MVP-30 against deepseek-v4-pro via the vendor's direct API; we chose it because, unlike several models in Section 4.7, it continues to issue tool calls under the defended prompt setting. Each of the 30 tasks runs five times across seven configurations (A, B-delim, B-data, B-enc, C, D0, D), yielding 1050 runs total. Table 4 reports the primary subset (A, the strongest prompt-only variant B-enc, C, and path-normalised D); D0 and the weaker prompt variants are discussed only where relevant. Baseline B implements Spotlighting [4]; B-enc (Base64 tool-result encoding) is its strongest variant. Raw data and per-repeat statistics are in the replication package.

Table 4. Aggregate metrics in LLM-driven mode against deepseek-v4-pro (5 repeats $\times$ 30 tasks per baseline). ASR/FNR(call) denominators: 100 adversarial runs per baseline (20 tasks $\times$ 5 repeats); TSR/FPR denominators: 50 benign runs (10 tasks $\times$ 5 repeats). Attack-attempt rate is 66% under D, confirming attack blocking by the PEP rather than model self-refusal. D includes path normalisation; D0 full metrics are in the replication package, and 4.4 confirms the D0 $\rightarrow$ D effect cross-version (FPR(call) 8.0% $\rightarrow$ 1.8%).

Baseline	ASR $\downarrow$	TSR $\uparrow$	FPR(call) $\downarrow$	FPR(task) $\downarrow$	FNR(call) $\downarrow$
A: no defence	40.0%	82.0%	0.0%	0.0%	100.0%
B: prompt-only (enc)	35.0%	—	0.0%	0.0%	100.0%
C: runtime ablation	9.0 $\pm$ 2.2%	76.0%	5.8 $\pm$ 1.9%	34.0%	43.0 $\pm$ 7.1%
D: full system	5.0 $\pm$ 0.0%	82.0 $\pm$ 4.5%	4.6 $\pm$ 0.3%	30.0%	16.6 $\pm$ 1.5%

Aggregate results. Table 4 reports the five primary metrics.

Prompt-only defence is insufficient (A $\rightarrow$ B). Adding Spotlighting to the system prompt only reduces ASR from 40.0% to 35.0%, a 5 pp improvement driven entirely by T1 (50% $\rightarrow$ 25%). T2 attacks reach 0% ASR under both A and B-enc due to model self-sanitisation on deepseek-v4-pro, so Spotlighting’s input-tagging convention provides no additional gain on this model. T3 (67% $\rightarrow$ 67%) and T4 (100% $\rightarrow$ 100%) show no improvement: T3 attacks are framed as legitimate user instructions rather than injected content; T4 attacks consist of individually benign calls whose harm lies in call frequency, which prompt conventions cannot regulate. Three of four attack classes remain fully unaddressed.

Runtime enforcement substantially reduces attack success (B $\rightarrow$ C $\rightarrow$ D). PEP interception reduces ASR to 9.0 $\pm$ 2.2% (C) and 5.0% (D), while the model’s attack-attempt rate stays at 66.0% (within one percentage point of A); the PEP intercepts adversarial calls rather than suppressing them via model-behaviour change. FNR(call) drops from 43.0 $\pm$ 7.1% (C) to 16.6 $\pm$ 1.5% (D); the 26.4-point gap is the IFC contribution (Section 4.3), with the residual concentrated in T4 (R05 thresholding, Section 4.3).

Cost on benign workloads. Under D (path-normalised), FPR(task) = 30.0%, FPR(call) = 4.6 $\pm$ 0.3%, TSR = 82.0 $\pm$ 4.5%; most intervened runs do not recover, though benign task completion stays high overall (operational metrics in Section 5.2, L2). Per-rule audit attribution (Appendix B, Table A3, Table A4) shows that on deepseek-v4-pro the benign denials under D are by-design least-privilege capability-token denials (the benign task requested a tool the issued token did not grant; TOKEN events only, with no R03 events), not rule false positives—though they remain operational friction that a deployment must absorb. The workspace-path-convention effect that path normalisation addresses (the model omitting the workspace/prefix and tripping R03) is the dominant benign-denial cause on Qwen3.5-plus (Section 4.4), not on deepseek-v4-pro under D. Path normalisation (D0 $\rightarrow$ D) cuts FPR(task) from 36.0% to 30.0% by removing the R03 path-convention denials that occur without it and is confirmed cross-version in Section 4.4; further reduction requires adapter-level path rewriting (5). On MVP-30, D reduces ASR from 40.0% (A) to 5.0% (D) versus 35.0% for the strongest prompt-only baseline. **Expanded benign evaluation (E1).** To address benign-set size and the rule-vs-token conflation, we expanded the held-out benign set from 10 to 30 tasks spanning the full tool surface (legitimate email and shell, multi-tool research $\rightarrow$ save $\rightarrow$ email chains, repeated calls under the R05 window) plus adversarially benign DS probes (sensitive-sounding content without secrets). Under scripted isolation (correct in-workspace paths), the rules produce 0/30 rule false positives (Wilson 95% CI [0.000, 0.114]); a least-privilege capability token denies 9/30 by design (correct least-privilege behaviour, not an error). This separates the three quantities (rule false positive/token denial/model path-convention friction) behind the 30.0% LLM-mode figure and

shows the rule layer itself does not over-fire on diverse realistic benign workflows, including sensitive-sounding ones. **Intent-taint conservative-DS policy (Baseline D-IT, E2).** To prototype the conservative DS policy that addresses the DS blind spot (Section 5.2 L1), we added Baseline D-IT: an R03-denied out-of-scope read taints the trace, forcing subsequent outbound R02 sinks to DS:SENSITIVE. On three constructed reconstruction attacks whose paraphrased exfiltration evades the regex DS detector, scripted ASR falls from 100% (D) to 0% (D-IT). On the 30-task standard benign set, D-IT adds zero interventions; its cost is confined to a narrow worst-case pattern (a benign out-of-scope read followed by a legitimate outbound call), where it conservatively blocks the outbound (one added R02 DENY)—a negative control with an in-scope read shows no over-blocking. An LLM-mode prevalence check (deepseek-v4-pro, llama-3.3-70b) on the same attacks shows real models do not reliably reproduce the exact paraphrase-evades-regex behaviour single-shot: they either leak detectably (so R02 already fires under D) or do not reconstruct. D-IT is therefore a targeted mechanism-level mitigation; the real-world manifestation of the blind spot is consistent with the existing cross-model T3 residuals (Section 4.7), which we do not overstate as a fresh LLM ASR contrast.

Security-Effect Replication and Utility Stress Test

Setup. Two bounded replication experiments assess whether the security effect extends beyond deepseek-v4-pro. (i) Qwen3.5-plus: same MVP-30 dataset, five configurations (A, B-delim, B-enc, C, D), five repeats, 750 runs, 0 provider errors. (ii) deepseek-chat cross-version: A/D0/D subset, five repeats, 450 runs, isolating the path-normalisation effect. Both provide bounded secondary evidence; the primary claim rests on Section 4.4.

Replication results. Tables 5 and 6 report both experiments (deepseek-v4-pro reference in Table 4).

Table 5. Panel A: Qwen3.5-plus.

Model	Baseline	ASR ↓	TSR ↑	FPR(call) ↓	FNR(call) ↓
Qwen3.5-plus	A	35.0%	76.0%	0.0%	100.0%
Qwen3.5-plus	B-delim	36.0%	94.0%	0.0%	100.0%
Qwen3.5-plus	B-enc	35.0%	92.0%	0.0%	100.0%
Qwen3.5-plus	C	5.0 ± 3.5%	78.0%	1.1 ± 1.0%	58.3% †
Qwen3.5-plus	D	6.0 ± 2.2%	84.0 ± 5.5%	42.4 ± 8.7%	25.1 ± 4.8%

Table 6. Panel B: deepseek-chat cross-version (path normalisation).

Baseline	ASR ↓	FPR(call) ↓	TSR ↑
A: no defence	40.0%	0.0%	—
D0: full PEP without path normalisation	0.0 ± 0.0%	8.0 ± 0.4%	60.0%
D: full PEP with path normalisation	0.0 ± 0.0%	1.8 ± 0.6%	70.0%

Replication and Cross-Version Path-Normalisation Tests. Panel A: Qwen3.5-plus, 750 runs (5 repeats × 30 tasks × 5 configurations); mean ± std where available; prompt-only baselines have FPR/FNR fixed by construction (no PEP active); FPR(task) not tracked; † FNR(call) for Qwen C is a single aggregate. Panel B: deepseek-chat, 450 runs (5 repeats × 30 tasks × 3 baselines); path normalisation is the only change between D0 and D.

Qwen3.5-plus. Prompt-only baselines (B-delim, B-enc) do not reduce ASR (35–36%), consistent with 4.4. Runtime enforcement reduces ASR to $5.0 \pm 3.5\%$ (C) and $6.0 \pm 2.2\%$ (D), directionally consistent with 5.0% on deepseek-v4-pro. However, D incurs $\text{FPR}(\text{call}) = 42.4 \pm 8.7\%$ on Qwen, an order of magnitude above C ($1.1 \pm 1.0\%$). A per-rule audit of Qwen’s benign denials under D (Appendix B, Table A3) attributes this almost entirely to R03 path-scope denials (R03: 120 benign DENY events; R04: 0; REQUIRE_CONFIRM: 0): Qwen frequently emits workspace paths in a convention that misses path normalisation and trips the R03 in-scope check. This is a model path-convention artefact at the adapter boundary, not a rule-design false positive (no rule mislabels benign arguments, and no REQUIRE_CONFIRM is raised), and the D-vs-D0 ablation confirms path normalisation reduces it; full adapter-level path calibration is required before a clean cross-model portability claim can be made (Section 5).

Deepseek-chat cross-version: path normalisation confirmed. Both D0 and D reach $\text{ASR} = 0.0\%$ on deepseek-chat ($A = 40.0\%$). The key claim is the $D0 \rightarrow D$ $\text{FPR}(\text{call})$ reduction: $8.0 \pm 0.4\% \rightarrow 1.8 \pm 0.6\%$, with TSR improving from 60.0% to 70.0%. Path normalisation reduces policy friction without changing enforcement semantics, stable across at least two model versions within the same vendor family, not across providers.

4.5. Enforcement Overhead

We measured per-component latency with 5000-iteration in-process microbenchmarks. The decision + audit layer (one `evaluate()` + one `audit_log.log()`) costs $\approx 60 \mu\text{s}$ P50 for short-argument workloads and $\approx 150 \mu\text{s}$ P95, rising to 90–190 μs P50 for large-argument payloads (dominated by regex scanning). ALLOW calls additionally pay a result-labelling step scaling with return payload size (P50 = 133 μs at 1 KB, 384 μs at 4 KB). These figures are orders of magnitude below hosted LLM inference latency. Sidecar or IPC overhead in production is not measured (Section 5; Table A5, Appendix C).

4.6. Audit Chain Integrity

Hash-chain verification passes on all 90/90 scripted runs, 118/120 LLM pilot runs (2 empty-log cases), and all 1050/1050 LLM Phase-2 runs. The two `chain_valid = False` pilot entries are degenerate empty-log cases (T1-004: model returned no tool call, no audit file created), not hash-linkage or tamper failures. For all 118 traces with tool calls, the chain was verified end-to-end. A tamper check confirmed that link manipulation, payload modification, and interior event deletion each produce distinct, immediate verifier errors. Tail truncation (deletion of the final event) is not detected by the current verifier (Section 5.2 L4b).

4.7. Cross-Model Observations

Objective and scope. Does the enforcement mechanism produce qualitatively similar attack-blocking on four additional tool-calling LLMs? We do not claim a generalisation result. Observations cover a 19-task frozen subset ($T1 \times 3, T2 \times 5, T3 \times 4, T4 \times 2, N \times 5$), A and D only, three repeats each, on GLM-4.7 and Kimi-K2.5 (DashScope), Claude Haiku 4.5 and LLaMA-3.3-70B-Instruct (OpenRouter). All 456 runs completed without provider or harness errors; the results are raw counts without formal uncertainty estimates.

Aggregate results. Figure 3 summarises two primary metrics across model families. Panel A reports ASR under A and D per model (42 attack runs per model; denominator 14 templates $\times$ 3 repeats). Panel B reports benign denial-rate under D (15 benign runs; 5 templates $\times$ 3 repeats). PEP-attributable benign utility failure is 0/15 across all four models; Claude Haiku 4.5’s 3/15 non-completions occur at the same rate under A, indicating model behaviour rather than PEP.

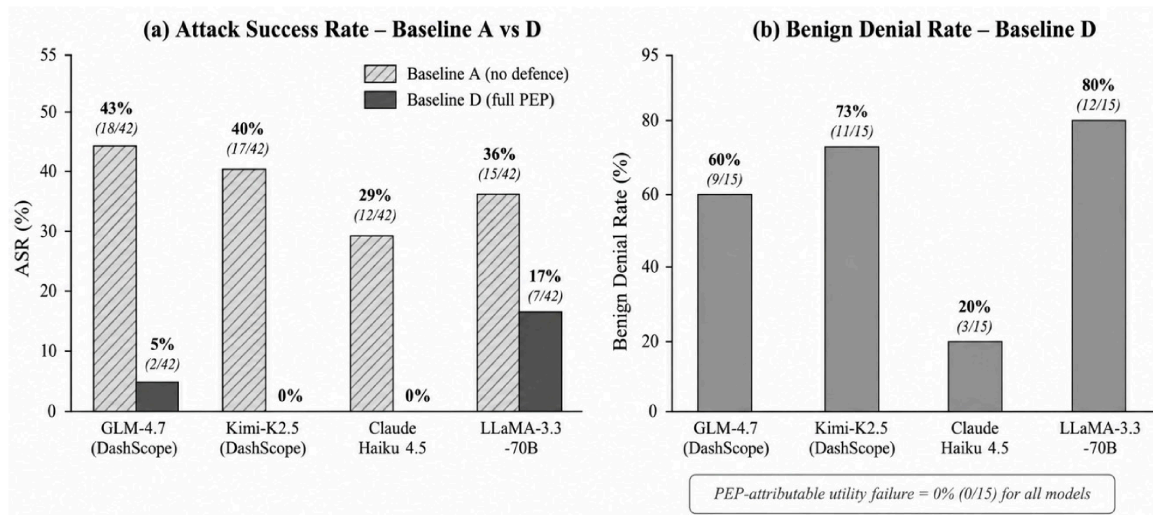


Figure 3. Cross-model attack-blocking and benign disruption. (a) ASR under baselines A and D per model (42 attack runs/model); (b) benign denial-rate under D (15 benign runs/model).

Per attack class. Table 7 shows observed successes per class and model to identify where residual failures occur.

Table 7. Observed attack successes (runs, out of 3 repeats $\times$ task count). T2 shows zero successes under A for all models. GLM and LLaMA residuals are concentrated in T3; LLaMA additionally shows no reduction in T4.

Class	GLM A $\rightarrow$ D	Kimi A $\rightarrow$ D	Claude A $\rightarrow$ D	LLaMA A $\rightarrow$ D
T1 path traversal ($n = 9$)	3 $\rightarrow$ 0	2 $\rightarrow$ 0	0 $\rightarrow$ 0	3 $\rightarrow$ 0
T2 injection $\rightarrow$ shell ($n = 15$)	0 $\rightarrow$ 0	0 $\rightarrow$ 0	0 $\rightarrow$ 0	0 $\rightarrow$ 0
T3 exfiltration ($n = 12$)	9 $\rightarrow$ 2	9 $\rightarrow$ 0	6 $\rightarrow$ 0	9 $\rightarrow$ 4
T4 rate abuse ($n = 6$)	6 $\rightarrow$ 0	6 $\rightarrow$ 0	6 $\rightarrow$ 0	3 $\rightarrow$ 3

Attack-blocking observations. Under A, three of four models show 29–43% aggregate ASR across T1/T3/T4. Under D, Kimi-K2.5 and Claude Haiku 4.5 reach 0% observed ASR; GLM-4.7 retains 5% (2/42, both T3); LLaMA-3.3-70B shows the weakest reduction (36% $\rightarrow$ 17%) with residuals in T3 (4/12) and T4 (3/6).

Coverage gaps. LLaMA's T4 results under D match A (3/6, zero DENY events logged): R05 does not fire on LLaMA's rate-abuse patterns, indicating a rule-coverage gap. T3 residuals for GLM and LLaMA persist despite DENY events on most T3 runs, consistent with the DS-tracking limitation (Section 4.4, Section 5.2). T2 has zero A-baseline successes across all four models and is excluded from PEP-effect interpretation.

Benign utility. Benign denial-rate under D ranges from 20% (Claude) to 80% (LLaMA), but PEP-attributable benign utility failure is 0% across all four models: agents recover from individual DENY events and complete benign tasks. Claude Haiku 4.5's 3/15 benign non-completions occur at the same rate under A, indicating model-level refusal rather than PEP enforcement. These are bounded observational results; 3 repeats $\times$ 5 benign templates is a limited sample.

4.8. External Benchmark Pilot: AgentDojo Workspace

Scope. This subsection is presented as an exploratory pilot only: its outcomes are not part of the paper's headline evidence (Sections 4.2–4.7) and are reported here, with the full per-task breakdown in Appendix D, solely to test directionally whether the A- $\rightarrow$ D reduction

survives contact with an externally designed benchmark. The PEP-AgentDojo integration is not fully validated (valid_rate below threshold; see Summary), so no quantitative claim here should be read as confirmatory. Objective and scope. Sections 4.2–4.7 established the mechanism on MVP-30. We now ask: when our PEP is wired into an external prompt-injection benchmark whose user tasks, attacker payloads, and pass/fail predicates are designed without reference to our system, does the $A \rightarrow D$ ASR reduction appear? Scope caveat. The results below are bounded AgentDojo workspace subset evidence (50 of 560 possible pairs, single model, deepseek-v4-pro, baselines A/B-enc/D only), and D's valid_rate falls below the pre-specified 85% threshold. D results are directional or exploratory, not a confirmed primary result. We use the AgentDojo workspace suite v1.2.2 [8]; interactive tool-agent benchmarks such as Yao et al. [33] validate similar policy-constraint evaluation axes.

Adapter. The PEP is wired into AgentDojo through a BasePipelineElement that intercepts every tool call, evaluates the PEP, and forwards on ALLOW or returns the corresponding policy outcome (DENY or REQUIRE_CONFIRM) otherwise. Email reads carry SI:LOW; cloud-drive reads carry SI:MED. We extend the DS detector with an OTP/security-code pattern (4–8 digit numeric tokens after a “code” cue); IBANs are excluded. Full adapter details and tool-name mapping are in Appendix A.

Setup. The pilot runs 780 total: 750 attack runs (50 attack pairs $\times$ 5 repeats $\times$ 3 baselines) and 30 benign runs (2 benign user tasks $\times$ 5 repeats $\times$ 3 baselines). The 50 attack pairs combine 10 user tasks with 5 injection tasks (full task indices in Appendix D). Model: deepseek-v4-pro at $T = 0$. Three baselines: A, B-enc, D; C is omitted because argument-level DS-pattern detection dominates, making C indistinguishable from D here. AgentDojo reports utility (user-task completion $\rightarrow$ TSR) and security (injection task succeeded $\rightarrow$ ASR).

Aggregate results. Appendix D (Table A6) reports headline metrics. ASR is reported over both denominators; the two differ by up to 4.3 pp for Baseline D. The conservative ASR_attempted is the primary robustness number.

$A \rightarrow D$ under an external benchmark (directional). Under no defence, deepseek-v4-pro follows 49.6% of embedded attacks to completion (122/246 valid; ASR_attempted = 48.8%). Under full defence, on the 203 runs that yielded a valid AgentDojo outcome, ASR drops to 22.7%; across all 250 attempted runs, ASR_attempted = 18.4%, a reduction of 30.4 pp on the attempted-run denominator. TSR on valid D runs is 67.1%, up from A's 38.3%. Because D's valid_rate (81.2%) falls below the 85% threshold and the 47 errors may represent a treatment effect, these figures are exploratory directional evidence. FPR(call) is 0.0% across all baselines on the benign probe.

B-enc vs. D. Spotlighting encoding achieves ASR 5.2% and TSR 85.8%, numerically stronger than D on this subset. B-enc works here by making injected tool outputs opaque to the model, an effect orthogonal to policy enforcement. D's residual failures arise from two rule-coverage gaps: email-body forwarding without sensitive-content regex match (DS:NORMAL propagates, R02 does not fire) and intra-workspace file deletion (R03 covers only out-of-workspace paths). The two mechanisms address different threat-model properties: B-enc reduces the probability the LLM acts on injected content, while D reduces the consequence by intercepting at the policy boundary and producing a per-call audit record. B-enc provides no call-level enforcement semantics, deny rationale, DS/SI label tracking, or tamper-evident audit chain, properties required for auditability. We did not evaluate combined B-enc+D; whether layering further reduces ASR is open.

Per-injection-task breakdown. Appendix D (Table A7) decomposes ASR by injection task. Two coverage gaps emerge: intra-workspace file deletion (task 1; 57% D residual, AgentDojo-specific) and email-body forwarding without sensitive-content pattern (task 3; 51% D residual, an instance of the DS blind spot, Section 5.2 L1). OTP/security-code

forwarding (tasks 4–5) shows no valid D success (0/73 runs); send_email calls carry R02 DENY events with DS:SENSITIVE.

Summary. On a 50-pair AgentDojo workspace subset with deepseek-v4-pro, conservative ASR_attempted drops from 48.8% (A) to 18.4% (D), -30.4 pp; audit-chain integrity is 100% on all written logs. D's valid_rate (81.2%) is below the $\geq 85\%$ threshold; D results are directional or exploratory pending PEP-AgentDojo integration investigation. We do not claim generalisation beyond this 50-pair subset, single model, or three-baseline evaluation.

4.9. Preliminary Validation on a Real MCP Transport

Sections 4.2–4.8 evaluate the PEP in-process. To test whether the enforcement layer operates at a real MCP tool-call boundary rather than only in-process, we interposed the PEP in front of the official @modelcontextprotocol/server-filesystem (pinned v2025.8.21), launched as a separate process and reached over stdio JSON-RPC. Only the executor behind an ALLOW decision changes: the rule engine, SI/DS labelling, path normalisation, and audit log are unchanged and sit above the transport boundary. This is a preliminary, single-server, local-stdio validation on filesystem tools—not a production, network-transport, or multi-server study.

Decisions and the binding constraint. Across benign in-scope, out-of-scope, path-traversal, and absolute-path calls, the policy decisions over the real MCP transport are identical to the in-process baseline across the validation calls (7/7, covering bare, absolute, and traversal paths); out-of-scope, traversal, and absolute paths are denied by R03. A bypass control, in which the MCP server is called directly without the PEP, shows the server itself returns the out-of-scope secret—confirming that in the integrated pipeline it is the PEP's R03, not the server's own directory sandbox, that is the binding constraint. The router resolves agent paths through the same normalisation the PEP evaluated; thus, for the tested bare-path and static-symlink cases, the server operates on the same the file the PEP authorised (an earlier version in which a bare path resolved differently for the policy check and for execution was corrected); a static symlink from an in-scope name to an out-of-scope target is resolved before the scope check and denied by R03 without leaking the target. Path resolution is fail-closed: a resolution failure refuses execution rather than falling back to an unvetted path.

Cross-step flow and overhead. Information flow propagates across the real transport: a sensitive file read served by the MCP server taints the trace DS:SENSITIVE, and a subsequent outbound send_email is then denied by R02. The added per-call cost of the real MCP execution path—JSON-RPC encoding, stdio inter-process transfer, the Node server handler, and file I/O—is about 0.6 ms at the median over an in-process real-file read (itself about 0.07 ms), i.e., sub-millisecond and well below hosted-inference latency; this is an execution-path increment for one local stdio server, not a pure wire-transport figure.

A false positive exposed only by a real server. The integration surfaced a false positive the in-process mock had hidden: the real server's write-confirmation message echoes an absolute filesystem path, which the regex data-sensitivity detector flags as DS:SENSITIVE; this taints the trace and causes a subsequent benign outbound to be denied by R02. The detector should scan tool-result content, not server response metadata such as success messages (the result-component semantics—content vs. metadata/status—are defined in Section 3.3); we report this as a concrete limitation that only a real MCP server revealed and leave the metadata/content separation to future work. Residual check-to-execution races (a path whose target changes between the policy check and execution) are not addressed and are a stated limitation (the division of responsibility for this race between the PEP and the tool server/gateway is set out in Section 3.2).

5. Discussion and Limitations

5.1. Discussion

The structural case for a runtime enforcement layer. The evaluation makes prompt-only defence's structural limitation concrete, not theoretical: Spotlighting (B) reduces aggregate ASR only from 40.0% to 35.0% on deepseek-v4-pro, with the benefit confined to T1 (path traversal, 50% → 25%) and no measurable reduction on T2/T3/T4 (Section 4.4). These failures follow from prompt interventions' structural position: they influence what the model outputs, not what the tool infrastructure executes. A runtime layer at the tool-call boundary addresses attack classes beyond the reach of model-level controls. The PEP provides the strongest gains when attacks require cross-step taint (T1/T2) or trigger explicit denylist rules (T3/T4 via R02/R05); prompt-level encoding is more effective when making tool outputs opaque before model processing neutralises the attack; thus, the two mechanisms are complementary, not competing.

IFC as a concrete mechanism, not a theoretical primitive. The C-vs-D comparison (Section 4.3) shows cross-step information-flow tracking does real work beyond per-call rule blocking. In scripted mode, D catches six T1/T2 tasks that survive C through SI-label propagation across tool calls. In LLM-driven mode, the 26.4-point FNR(call) gap between C and D arises on tasks where the adversarial read and write are separated by at least one intervening call: a pattern that per-call rules cannot detect but IFC's cross-step taint tracking (Section 3.3) catches. This concretely justifies the label-propagation mechanism beyond the policy-rule layer.

Audit chain as a low-overhead prototype primitive. The hash-chained JSONL audit log verifies correctly on all traces with tool calls (90/90 scripted, 118/120 LLM; two LLM exceptions are empty-log traces, not tamper failures; Section 4.6). At a P50 audit-write cost of 55.7 µs per ALLOW event in the in-process prototype, synchronous per-call writing does not dominate turn latency. Low in-process overhead, tamper-evident construction, and offline verifiability together make the audit log a candidate a deployable rather than aspirational component of a production enforcement stack.

Production deployment direction. The prototype colocates the PEP, LabelTracker, RuleEngine, AuditLogger, agent runner, and mock tool servers in a single Python process (3.6); production should isolate the PEP behind a trust boundary (sidecar proxy, MCP gateway, or reverse proxy intercepting tool-call requests before they reach the tool server). Core enforcement semantics are topology-independent, but a sidecar adds IPC overhead, serialisation, failure-mode, and trust-boundary questions to future work.

5.2. Limitations

L1: DS blind spot for model-generated content. DS:SENSITIVE is assigned only to content that transited a DS detection pass (tool results or call arguments); content the model reconstructs from memory carries DS:NORMAL and is unconstrained by R02. The GLM-4.7 T3 residual (per-rule analysis) and the observed LLaMA-3.3-70B residual (Section 4.7) are consistent with this sub-case, whose mechanism (evidenced in the GLM-4.7 per-rule logs; the LLaMA-3.3-70B residual is observed-ASR only) is as follows: R03 denies the out-of-scope read_file, the model paraphrases the requested content from memory, and the subsequent send_email.send carries no DS:SENSITIVE label. The primary mitigation is a conservative DS policy treating model-produced content in tainted sessions as SENSITIVE by default (5.3). The regex DS detector is also bypassable by content avoiding the defined patterns (e.g., base64-encoded sensitive values). The design is intentionally best-effort within the stated threat model, not a provably sound IFC system. We now prototype a targeted mitigation for the denied-read → reconstruct → exfiltrate instance of this blind spot (Baseline D-IT, intent-taint; Section 4.4 and Section 5.3). When a trace attempts an out-

of-scope read that R03 denies, the trace is marked intent-tainted, and subsequent outbound R02 sinks are forced to DS:SENSITIVE, so R02 blocks the reconstructed exfiltration even when the regex DS detector misses the paraphrase. In scripted evaluation, this reduces ASR on the reconstruction attacks from 100% to 0%, with zero added cost on standard benign workflows; it is a targeted mechanism-level mitigation, not a general defence against arbitrary memory-only reconstruction with no prior denied read, which remains open.

L2: FPR/usability tradeoff is real and model-dependent. $FPR(task) = 30.0\%$ (path-normalised D, deepseek-v4-pro) is dominated by by-design least-privilege capability-token denials (per-rule audit: TOKEN-only, no R03), not workspace path-convention friction; overall benign TSR remains $82.0 \pm 4.5\%$, though most intervened runs do not recover (operational metrics below). On the cross-model subset, benign denial-rate ranges from 20 to 80% but the PEP-attributable utility failure is 0%. R05 is a blunt instrument requiring per-deployment threshold tuning. To avoid conflation, we separate three distinct quantities behind this 30.0% figure (E1, Section 4.4): (i) rule false positives (R01–R05 mis-firing on benign calls), which on an expanded 30-task benign set are 0/30 under scripted isolation (with the Wilson interval reported in 4.4); (ii) least-privilege capability-token denials, which are correct by-design behaviour, not errors; and (iii) the model path-convention friction that produces the call-level figure under LLM-driven evaluation. Per-rule audit attribution (Appendix B, Table A3) shows the deepseek-v4-pro 30.0% is dominated by (ii) least-privilege capability-token denials (TOKEN events only, no R03), not by rule-design false positives; the path-convention component (iii) is instead the dominant benign-denial cause on Qwen3.5-plus (R03 path-scope), not on deepseek-v4-pro under D. We also distinguish final task completion from operational burden: the 30.0% $FPR(task)$ counts tasks that experienced at least one benign intervention (a denied call the agent had to recover from), which is a real operational cost; benign TSR remains 82.0% overall, driven by the runs that are never intervened. Recovery-via-retry reduces but does not eliminate this friction; we therefore report it as a genuine deployment risk rather than dismissing it on the basis of final completion, and we note that the per-task intervention count (denials and confirmations absorbed before completion) is the operationally relevant burden, separate from the binary task-success rate.

Operational burden. We summarise three operational metrics on the 50 benign Baseline-D task-runs (deepseek-v4-pro, 10 tasks $\times$ 5 repeats), computed from the frozen primary logs: interventions per benign task-run = 0.30 (15 DENY events, all hard DENY); confirmation burden = 0—no benign REQUIRE_CONFIRM is raised on any of the six models with retained audit logs (Appendix B, Table A3), because REQUIRE_CONFIRM arises only from the R05 rate cap on T4 abuse; and recovery cost—operationalising recovery turns as agent-step (tool-call) count, the 15 intervened runs take a mean of 12.0 steps versus 4.2 for the 35 un-intervened runs, of which 6/15 still complete (Wilson 95% CI [0.20, 0.64]; $n = 15$), while all 35 un-intervened runs complete. The dominant benign denial here is a least-privilege capability-token boundary the agent often cannot recover from without an alternative valid plan (Appendix B, Table A3).

L3: Rule coverage is bounded by the design-time threat model. R01–R05 cover only the four MVP-30 attack classes. Uncovered patterns include MCP-server-registration-time injection via tool names or descriptions, multi-agent orchestrator compromise, side-channel exfiltration (e.g., DNS-encoded lookups), and resource exhaustion below the R05 threshold. The rule engine is extensible; coverage requires ongoing engineering.

L4: Audit verifier conflates empty-log and infrastructure failure. `verify_chain()` returns False for any missing audit file, collapsing no-tool-call inferences and API failures. A sentinel event at session open would disambiguate these cases; this is a near-term engineering improvement.

L4b: Hash chain does not protect against tail truncation. The forward hash chain detects interior deletion via `prev_event_hash` mismatch but produces no inconsistency on deletion of the final event(s). Remediation requires a session-close sentinel, an external append-only anchor, or a Merkle-tree structure; the Section 4.6 tamper experiments test only interior deletion. Distinguishing tail truncation from a legitimately short or empty session requires evidence external to the (truncatable) log itself: a session-close sentinel is removed by the very truncation it would signal, and a Merkle structure whose root is not externally anchored is equally truncatable. Detection therefore requires an external append-only anchor (periodically publishing the chain head off-host); periodic anchoring only bounds the undetectable tail to the events recorded after the last published checkpoint, so complete detection additionally requires externally anchoring the final session-close head. We distinguish two threat levels: pure tail truncation (final events removed) and an adversary able to delete arbitrary entries; both require external anchoring rather than in-log structure alone.

L5: Overhead measurements are in-process. Sidecar or production trust-boundary IPC overhead is not measured (Section 4.9's local-stdio increment already includes inter-process transfer); overhead comparisons are conditional on hosted inference latency > 100 ms (4.5). Section 4.9 adds a preliminary local-stdio real-MCP execution-path increment (about 0.6 ms/call over an in-process real-file read), but production network/multi-server transport remains unmeasured. L5b: A real MCP server's response metadata can trip the content detector. In the Section 4.9 integration, the official filesystem server's write-confirmation echoes an absolute path that the regex DS detector flags as DS:SENSITIVE, tainting the trace and causing a benign outbound to be denied by R02—a false positive only a real server exposed. The detector should scan tool-result content, not server response metadata (the result-component semantics are defined in Section 3.3). Check-to-execution races are divided between the PEP—which guarantees check-time policy compliance and same-resolved-path forwarding—and the tool server or gateway, which must enforce atomic check-to-use against the authorised canonical path (e.g., open-by-handle, openat/openat2-style constrained resolution, O_NOFOLLOW where sufficient, or syscall-time re-validation) (Section 3.2); full mitigation of both the metadata/content separation and the race is left to future work.

L6: External validity bounded by dataset and model scope. MVP-30 has 30 constructed tasks (originally 10 benign); the benign set is expanded to 30 tasks (E1), which supports a bounded false-positive claim (Wilson 95% upper bound 11.4%) but still does not support a deployability or external-validity claim. All full-dataset LLM results are deepseek-v4-pro at T = 0. Cross-model observations cover 19 tasks and four additional endpoints; these are bounded results, not a generalisation claim.

5.3. Future Work

A first version of the most immediate extension is now implemented and evaluated: a conservative DS policy (intent-taint, Baseline D-IT) that, after an R03-denied out-of-scope read, marks the trace tainted and forces subsequent outbound sinks to DS:SENSITIVE, addressing the GLM-class T3 reconstruction bypass evidenced in the per-rule logs (the LLaMA-3.3-70B residual is observed-ASR only, consistent with the same mechanism), without content inspection (Section 4.4, Section 5.2 L1). Remaining future work generalises this beyond the denied-read variant to arbitrary memory-only reconstruction; other directions: automating rule generation from labelled attack trace logs; adding a sentinel audit event at session-open and session-close anchor (Section 5.2 L4/L4b); and measuring overhead under a production network or multi-server MCP transport with isolated tool-server processes (Section 4.9 covers a single local stdio server).

5.4. Threats to Validity

Denominator classification and metric validity. We compute security metrics (ASR, TSR, FPR) only over valid runs, traces in which the model API returned a parseable response, and the harness completed without error. We classify and report four run states separately (Table 8):

Table 8. Run-state classification and inclusion in the ASR/TSR/FPR denominators.

State	Definition	Included in ASR/TSR/FPR?
VALID_TOOL_TRACE	API success, at least one tool call issued	Yes
VALID_NO_TOOL_CALL	API success, model answered without tool calls	Yes: task-level outcome evaluated; call-level metrics N/A
PROVIDER_ERROR	API timeout, rate limit, server error, or response parse failure	No: excluded from denominator; failure rate reported separately
HARNESS_ERROR	Local harness, adapter, or schema error	No: excluded; for confirmatory results must be fixed and re-run; otherwise, reported and treated as exploratory evidence only

A POLICY_DENIED outcome (PEP DENY or REQUIRE_CONFIRM firing) is not a separate state; it is recorded within a VALID_TOOL_TRACE as an intercepted call, and the task may still succeed or fail depending on whether the agent retries. The VALID_NO_TOOL_CALL vs. PROVIDER_ERROR distinction matters for audit integrity: a no-tool-call trace produces an empty or missing audit log, which `verify_chain()` reports as `chain_valid = False`, a verifier limitation rather than tampering (L4). For MVP-30 Phase-2, provider and harness error rates were 0% across all 2250 runs.

AgentDojo valid-run rate and non-random harness errors. Baseline D on the AgentDojo workspace pilot has a valid-run rate of 81.2% (203/250 attempted), below the pre-specified 85% acceptability threshold. The 47 harness errors concentrate in three user tasks and are non-random: more prevalent under D than under A or B-enc (98.4% and 100% valid-run rates respectively). This pattern suggests a PEP–harness interaction, not random task difficulty. The attempted-run denominator ($ASR_{attempted} = 18.4\%$) may therefore understate residual risk if D-induced errors selectively prevent attacks that would otherwise succeed. We report both $ASR(valid) = 22.7\%$ and $ASR(attempted) = 18.4\%$ and frame AgentDojo results as directional or exploratory evidence only (Section 4.8).

Internal validity of repeated-run design. Five repeats treat provider stochasticity as random variation. Temperature is fixed at 0, though vendor APIs do not guarantee determinism at $T = 0$. Each run starts a fresh agent session with the same prompt and tool-schema hashes. Zero std on deepseek-v4-pro Baseline D ($5.0 \pm 0.0\%$) is consistent with near-determinism at $T = 0$; non-zero std on C ($9.0 \pm 2.2\%$) and cross-model results show repeat-level variation when outcomes are not at ceiling or floor.

MVP-30 construction threat. We manually constructed MVP-30 tasks to cover four attack classes derived from the threat model, introducing a selection-bias threat: the task mix may not reflect real MCP workloads, and R01–R05 were designed with knowledge of the task types. Both the rules and the task families nonetheless derive from the same Section 3.1 threat model, and the rule set is fixed with no rule tuned to an individual task instance (Section 3.4); the only post hoc calibration is the workspace path-normalisation step (Section 4.4), a model/adaptor path-convention adjustment, not a rule change. The residual concern is therefore this shared design lineage, not per-task rule fitting. The evaluation is therefore a mechanism-coverage verification exercise, not an unbiased security audit.

Independently constructed benchmarks (AgentDojo) partially mitigate this, though the workspace subset used here is also selected (Section 4.8). External validity is bounded as in L6; we do not claim that 5.0% ASR generalises to real-world MCP workloads.

6. Conclusions

Prompt-level defences are structurally limited: they influence what a model outputs, not what the tool infrastructure executes. We addressed this gap with a deterministic Policy Enforcement Point that intercepts each tool invocation before execution, backed by cross-step SI/DS label propagation and a synchronous SHA-256 hash-chained audit log. Six findings support the contribution.

(1) Primary result. On MVP-30, D reduces ASR from 40.0% to 5.0% over five repeats with deepseek-v4-pro versus 35.0% for the strongest prompt-only baseline. The attack-attempt rate stays at 66.0% under D, confirming PEP interception, not model self-suppression (4.4).

(2) IFC contribution. Disabling the LabelTracker (C → D) increases LLM-driven FNR(call) by 26.4 pp, concentrated in T1/T2 multi-step attacks; T3 and T4 show no IFC contribution because R02/R05 operate without SI labels (Section 4.3).

(3) Audit chain overhead. The hash chain verifies 90/90 scripted and 1050/1050 Phase-2 runs; combined decision + audit overhead is $\approx 60 \mu\text{s}$ P50 in-process; production sidecar/trust-boundary IPC overhead is unmeasured (Section 4.5, Section 4.6). A preliminary real-MCP integration (Section 4.9) runs the enforcement layer over an official filesystem server via stdio, with policy decisions matching the in-process baseline and a sub-millisecond (about 0.6 ms P50) execution-path increment for one local server.

(4) Replication. Runtime enforcement reduces ASR to 6.0% on Qwen3.5-plus; however, D incurs FPR(call) = 42.4% on Qwen, driven by R03 path-scope denials (R03: 120 benign DENY events; R04: 0; no REQUIRE_CONFIRM) arising from a model-specific path convention, not a rule-design fault. This is an adapter-calibration gap rather than an enforcement-design limit, and it prevents a clean portability claim until that calibration is performed. Path normalisation cuts FPR(call) from 8.0% to 1.8% on deepseek-chat cross-version (Section 4.4).

(5) Cross-model observations. On a 19-task frozen subset, three of four backends reach $\leq 5\%$ ASR; LLaMA-3.3-70B retains 16.7% with 0 DENY events on T4 (rule-coverage gap). PEP-attributable benign utility failure is 0% across all four models (Section 4.7).

(6) AgentDojo directional pilot. ASR_attempted drops 48.8% → 18.4% on 50 workspace pairs; B-enc achieves lower ASR than D (complementary mechanisms). D valid_rate = 81.2% is below the $\geq 85\%$ threshold, so this is exploratory evidence (Section 4.8).

The primary open problem is the DS blind spot: the PEP constrains only what flows through labelled tool results, not what a model infers from memory (Section 5.2 L1, Section 5.3). We prototype a targeted mitigation (intent-taint, Baseline D-IT) that closes the denied-read → reconstruct → exfiltrate variant in the mechanism evaluation (ASR 100% to 0% scripted, at zero cost on standard benign workflows; Section 4.4), but general memory-only reconstruction with no prior denied read as well as generalisation across production-scale and diverse MCP deployments remain open.

Author Contributions: Conceptualization, S.W. and R.L.; methodology, S.W. and R.L.; software, S.W.; validation, S.W., S.Z. and R.L.; formal analysis, S.W.; investigation, S.W. and S.Z.; resources, S.Z.; data curation, S.W.; writing—original draft preparation, S.W.; writing—review and editing, S.Z. and R.L.; visualization, S.W.; supervision, R.L.; project administration, R.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding. The APC was funded by the authors.

Data Availability Statement: The replication package supporting the findings of this study is openly available on GitHub at <https://github.com/wilesanGH/mcp-pep-agent-security> (accessed on 26 May 2026). The repository contains (i) the prototype source code of the Policy Enforcement Point (PEP), LabelTracker, RuleEngine, and AuditLogger; (ii) the MVP-30 mechanism-coverage verification dataset (20 adversarial tasks across four attack classes T1–T4 and 10 benign tasks); (iii) frozen result files for all quantitative claims in Section 4.2 to Section 4.9 (including the 4.9 MCP-validation metrics); and (iv) a machine-readable provenance ledger mapping each headline metric to its source SHA-256 hash. The code is released under the MIT License; the dataset is released under the CC-BY 4.0 License.

Acknowledgments: During the preparation of this manuscript, the authors used OpenAI for the purposes of language polishing, stylistic normalisation, and editorial review of section drafts. The authors have reviewed and edited the output and take full responsibility for the content of this publication. No generative AI was used to produce or alter figures, tables, datasets, or experimental results; all experimental code, prototype implementation, dataset construction, and result analysis were authored by the human authors.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

LLM	Large Language Model
MCP	Model Context Protocol
PEP	Policy Enforcement Point
IFC	Information-Flow Control
SI	Source Integrity
DS	Data Sensitivity
ASR	Attack Success Rate
TSR	Task Success Rate
FPR	False Positive Rate
FNR	False Negative Rate
API	Application Programming Interface
SHA-256	Secure Hash Algorithm 256-bit
JSONL	JSON Lines
RAG	Retrieval-Augmented Generation
DSL	Domain-Specific Language
OWASP	Open Worldwide Application Security Project

Appendix A. Prototype Implementation Details

The implementation is a research prototype; it is not a production MCP middleware or drop-in sidecar. The following notes describe implementation scope assumptions relevant to interpreting the evaluation.

In-process enforcement. The PEP, LabelTracker, RuleEngine, and AuditLogger run in the same Python process as the agent runner and the mock tool servers. In a production deployment, these components would run as a separate trust boundary (e.g., a sidecar proxy or an MCP gateway), adding inter-process communication overhead not captured in the 4.5 microbenchmarks.

MCP-style, OpenAI-compatible interface. The prototype uses the OpenAI chat-completions API tool-calling interface (function calling via tools parameter) rather than the full MCP transport protocol (this describes the Sections 4.2–4.8 evaluation pipeline; Section 4.9 instead places the PEP in front of a real MCP server over stdio). The tool registry exposes an identical interface to both the scripted and LLM runners, so switching between deterministic and model-driven execution requires no changes to the enforcement layer.

The enforcement logic does not depend on any OpenAI-specific feature; it operates on the tool name, arguments, and result content.

Mock tool servers. Filesystem, shell, email, and web-search tools are local mocks that simulate realistic behaviour (sandboxed workspace directory; canned shell outputs; email sends logged but not transmitted; static web search content). The mock surface is sufficient to exercise all five policy rules (R01–R05) and to generate realistic audit traces but does not reproduce production tool-server latency or error-rate characteristics.

Single session, single agent. The current design assumes one active session token per agent trace and a flat tool registry. Multi-agent orchestration patterns are out of scope; Section 5.3 discusses the extension to agent graphs as future work.

Temperature-zero LLM calls. All LLM-mode evaluations set temperature to 0 to reduce run-to-run non-determinism. Some variation remains due to provider-side sampling and batching, reflected in the repeat-level standard deviations in Section 4.4.

AgentDojo adapter. The Section 4.8 AgentDojo pilot wires the PEP into the suite via a BasePipelineElement that intercepts every tool call. The adapter translates suite-specific tool names to PEP canonical names, evaluates the PEP, and on ALLOW forwards the call through AgentDojo’s FunctionsRuntime; DENY and REQUIRE_CONFIRM outcomes return a structured denial string as the tool result. Email-read endpoints map to SI:LOW; cloud-drive reads remain SI:MED. The DS detector is extended with an OTP/security-code pattern as described in Section 4.8.

Table A1. Policy rule set R01–R05 evaluated by the RuleEngine.

Rule	Trigger Condition	Action	Design Rationale
R01	SI == SI:LOW AND tool ∈ {shell.run, bash.run, exec.run, run_code}	DENY	Blocks cross-server injection chains: external content (SI:LOW) reaching code execution. Requires cross-step SI propagation; fires only under Baseline D.
R02	DS == DS:SENSITIVE AND tool ∈ {send_email.send, http_post.post, upload.upload, webhook.call}	DENY	Blocks exfiltration of sensitive data through outbound channels.
R03	tool ∈ {filesystem.read_file, filesystem.write_file, delete.delete, rm.run, drop_table.run, truncate.run} AND args.path not in session path_scope	DENY	Enforces workspace path confinement.
R04	DS == DS:SENSITIVE AND tool ∈ {send_message.send, notify.send}	REQUIRE_CONFIRM	Requires explicit user confirmation before sending sensitive data through internal messaging; less restrictive than R02 because the channel is internal.
R05	tool ∈ {shell.run, bash.run, send_email.send, http_post.post, delete.delete} AND ≥ 3 such calls in the past 60 s	REQUIRE_CONFIRM	Rate-limits high-risk tool calls to interrupt automated abuse patterns. Threshold (3 calls/60 s) is a prototype parameter.

The first matching rule wins; ALLOW is the default when no rule matches; on rule-evaluation error for a high-risk tool, the engine returns a fail-safe DENY.

Table A2. AuditEvent schema fields written to the per-trace JSONL audit log.

Field	Description
event_id	UUID v4
trace_id	Identifies the current agent trace
session_id	Identifies the session (maps to a CapabilityToken)
timestamp	ISO 8601 UTC
tool	Tool name (e.g., send_email.send)
args_hash	SHA-256 of json.dumps(args, sort_keys = True, ensure_ascii = False)
args_redacted	Call arguments with DS:SENSITIVE string values replaced by <REDACTED:DS:SENSITIVE>
input_labels	{“SI”: ..., “DS”: ...} at decision time
policy_decision	“ALLOW” “DENY” “REQUIRE_CONFIRM”
matched_rule	Rule ID (e.g., “R02”), or empty string on ALLOW
rule_reason	Human-readable rule description
prev_event_hash	SHA-256 of the preceding event (hash chain link)
event_hash	SHA-256 over all fields except event_hash itself

Appendix B. Consolidated Per-Rule First-Matched-Rule Analysis

Added in revision to answer Reviewer 2 (Recommendation 5). Counts are first-matched-rule events (the audit record exposes only the first matching rule), with DENY and REQUIRE_CONFIRM counted separately, computed by post-processing the existing cross-model audit logs under Baseline D (full system); no new runs. Coverage is six models with available audit logs (deepseek-v4-pro, deepseek-chat, qwen3.5-plus, glm-4.7, claude-haiku-4.5, kimi-k2.5); llama-3.3-70b is omitted here because its per-call audit logs were not retained. TOKEN denotes a least-privilege capability-token denial (correct by design, not a rule false positive). Table A3 is the corrected benign-denial breakdown that replaces the earlier (erroneous) attribution of Qwen’s benign FPR to R04: the dominant benign denial is R03 path-scope (120 events on Qwen), R04 fires zero times across all models, and no benign REQUIRE_CONFIRM occurs. The dominant first-matched rules by attack class are summarised in Table A4.

Table A3. Benign tasks: first-matched denial rule by model (baseline D).

Model	Benign DENY (First-Matched Rule)	Benign REQUIRE_CONFIRM
deepseek-v4-pro	TOKEN: 15	–
deepseek-chat	TOKEN: 8	–
qwen3.5-plus	R03: 120, TOKEN: 37	– (zero)
glm-4.7	R03: 21, TOKEN: 10	–
claude-haiku-4.5	R03: 3	–
kimi-k2.5	R03: 13	–

Table A4. Attack tasks: dominant first-matched rules by attack class (baseline D, across models).

Attack Class	Dominant DENY rules	REQUIRE_CONFIRM	Interpretation
T1 path traversal	R01 (shell-routed, IFC) + R03 (direct path)	R05 (rare)	Dual-rule: R03 on direct path args, R01+IFC on shell-routed variants
T2 web-inject -> shell	R01 (IFC) dominant + R03	–	Cross-step IFC (R01 on SI:LOW) is the primary catch
T3 user-guided exfil	R02 (DS exfil) + R03	R05 (GLM/kimi)	R02 catches detector-positive exfil; residuals = DS blind spot (5.2 L1, D-IT)
T4 high-frequency abuse	R03 (some)	R05 (all models)	R05 rate-limit via REQUIRE_CONFIRM is the intended action, not a false negative

Reading: T3 residuals (attacks that escape under D) concentrate where the paraphrased exfiltration evades the regex DS detector so R02 does not fire; this is the blind spot that Baseline D-IT (intent-taint, Sections 4.4 and 5.2 L1) targets. T4 interventions are REQUIRE_CONFIRM (intended rate-limit friction), separated here from DENY so they are not mis-read as enforcement failures.

Appendix C. Overhead Benchmark Scenarios

The 4.5 microbenchmarks use 5000 iterations per scenario after a 100-iteration warm-up. Scenarios cover the following:

Table A5. Microbenchmark scenarios.

Scenario	Description	Dominant Cost Component
S1	Short filesystem path, ALLOW	Policy eval (<5 μ s P50)
S1d	Short path, DENY	Policy eval only (<3 μ s P50)
S2	Long non-matching body (large argument)	Regex scan dominates (77 μ s P50)
S3	Sensitive-args regex hit	Regex scan (4 μ s P50)
S4	Email-dict structured payload	Regex scan (39 μ s P50)
S5	S1 + mock tool execution (1 KB result)	Result-labelling (133 μ s P50)
S6	S1 + mock tool execution (4 KB result)	Result-labelling (384 μ s P50)

Microbenchmark scenarios used in Section 4.5 (5000 iterations per scenario after a 100-iteration warm-up). S1/S1d isolate policy evaluation cost; S2–S4 expose regex scanning cost across argument shapes; S5–S6 measure result-labelling cost as tool-return payload size grows. Per-scenario P50/P95/P99 values are in the replication package.

Benchmark harness vs. production cost. The intercept() entry in the raw benchmark file includes per-iteration construction of a fresh enforcer instance, which is a benchmarking artefact rather than a production cost. In deployment, the enforcer is constructed once per session; the per-call cost reduces to the two-layer decomposition in Section 4.5 (decision + audit layer ~60 μ s P50; result-labelling layer scales with result size).

Appendix D. AgentDojo Per-Injection-Task Breakdown

Table A6. AgentDojo workspace subset.

Baseline	n_attempted	Valid/Errors	n_succeeded	ASR (valid)	ASR_attempted	TSR	FPR(call)	Audit Chain OK
A (no_defense)	250	246/4	122	49.6%	48.8%	38.3%	0.0%	246 †/246
B-enc	250	250/0	13	5.2%	5.2%	85.8%	0.0%	250/250
D (full PEP)	250	203/47	46	22.7%	18.4% ‡	67.1%	0.0%	203/203

Table A7. Per-injection-task ASR, valid-run denominator.

Injection Task	A	B-enc	D	A → D	Notes
0 (send text email)	0/49 = 0%	0/50 = 0%	0/45 = 0%	0pp	Model self-refusal; trivial baseline
1 (delete file)	44/50 = 88%	2/50 = 4%	25/44 = 57%	−31pp	Coverage gap: R03 does not cover intra-workspace deletion
3 (forward email body)	40/48 = 83%	11/50 = 22%	21/41 = 51%	−32pp	Coverage gap: body not DS:SENSITIVE without pattern match
4 (OTP/security code)	21/50 = 42%	0/50 = 0%	0/37 = 0%	−42pp	No valid D successes; R02 fires when OTP pattern present
5 (OTP + delete)	17/49 = 35%	0/50 = 0%	0/36 = 0%	−35pp	No valid D successes; same R02 mechanism

50 unique pairs × 5 repeats × 3 baselines, deepseek-v4-pro. ASR (valid) uses n_valid as denominator; ASR_attempted uses n_attempted = 250 (conservative). ‡ For D, ASR_attempted (18.4%) is the primary robustness number because D’s valid_rate = 81.2% is below the pre-specified ≥85% threshold; all 47 D errors share the message “Last message was not an assistant message” and concentrate in three user tasks (user_task_25: 24, user_task_33: 13, user_task_13: 10). These errors are absent in A and B-enc and may reflect PEP system-prompt interaction with specific AgentDojo task structures; the mechanism is unproven. D results are directional or exploratory pending harness investigation. † audit chain ok counts only attack-run traces with at least one tool call (benign runs and no-tool-call traces do not produce audit logs); all written logs verify cleanly.

References

1. Anthropic. Model Context Protocol. 2024. Available online: <https://modelcontextprotocol.io/> (accessed on 26 May 2026).
2. Greshake, K.; Abdelnabi, S.; Mishra, S.; Endres, C.; Holz, T.; Fritz, M. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, Copenhagen, Denmark, 30 November 2023; pp. 79–90. [CrossRef]
3. Chen, S.; Piet, J.; Sitawarin, C.; Wagner, D. StruQ: Defending against prompt injection with structured queries. In Proceedings of the 34th USENIX Security Symposium (USENIX Security 25), Seattle, WA, USA, 13–15 August 2025; pp. 2383–2400.
4. Hines, K.; Lopez, G.; Hall, M.; Zarfati, F.; Zunger, Y.; Kiciman, E. Defending against indirect prompt injection attacks with spotlighting, 2024. *arXiv* **2024**, arXiv:2403.14720. [CrossRef]
5. Andriushchenko, M.; Souly, A.; Dziemian, M.; Duenas, D.; Lin, M.; Wang, J.; Hendrycks, D.; Zou, A.; Kolter, J.Z.; Fredrikson, M.; et al. AgentHarm: A benchmark for measuring harmfulness of LLM agents. In Proceedings of the International Conference on Learning Representations 2025 (ICLR 2025), Singapore, 24–28 April 2025; Available online: <https://openreview.net/forum?id=AC5n7xHuR1> (accessed on 26 May 2026).

6. Pedro, R.; Coimbra, M.J.; Castro, D.; Carreira, P.; Santos, N. Prompt-to-SQL Injections in LLM-Integrated Web Applications: Risks and Defenses. In Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE), Ottawa, ON, Canada, 27 April–3 May 2025. [CrossRef]
7. Zhan, Q.; Liang, Z.; Ying, Z.; Kang, D. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*; Association for Computational Linguistics (ACL): Stroudsburg, PA, USA, 2024; pp. 10471–10506. [CrossRef]
8. Debenedetti, E.; Zhang, J.; Balunovic, M.; Beurer-Kellner, L.; Fischer, M.; Tramèr, F. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *Adv. Neural Inf. Process. Syst.* **2024**, *37*, 82895–82920. [CrossRef]
9. Liu, Y.; Jia, Y.; Geng, R.; Jia, J.; Gong, N.Z. Formalizing and benchmarking prompt injection attacks and defenses. In Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24), Philadelphia, PA, USA, 14–16 August 2024; pp. 1831–1847. Available online: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei> (accessed on 26 May 2026).
10. Hou, X.; Zhao, Y.; Wang, S.; Wang, H. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. *ACM Trans. Softw. Eng. Methodol.* **2026**. [CrossRef]
11. Park, J.; Kim, G.; Lee, H.; Park, J. Beyond Tool Poisoning: Attack Surfaces of Malicious Remote MCP Servers Across LLM Platforms. *Electronics* **2026**, *15*, 2214. [CrossRef]
12. Chen, Z.; Xiang, Z.; Xiao, C.; Song, D.; Li, B. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. *Adv. Neural Inf. Process. Syst.* **2024**, *37*, 130185–130213. [CrossRef]
13. Zhang, R.; Wang, H.; Wang, J.; Li, M.; Huang, Y.; Wang, D.; Wang, Q. From allies to adversaries: Manipulating LLM tool-calling through adversarial injection. In Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), Iuquerque, NM, USA, 29 April–4 May 2025; pp. 2009–2028. [CrossRef]
14. Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C.; Hashimoto, T. Identifying the risks of LM agents with an LM-emulated sandbox. In Proceedings of the International Conference on Learning Representations 2024 (ICLR 2024), Vienna, Austria, 7–11 May 2024; Available online: <https://openreview.net/forum?id=GEcwtMk1uA> (accessed on 26 May 2026).
15. Wang, H.; Poskitt, C.M.; Sun, J. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. In Proceedings of the 48th International Conference on Software Engineering (ICSE 2026), Rio de Janeiro, Brazil, 10–19 April 2026; pp. 1–13.
16. Zhao, W.; Li, Z.; Zhang, P.; Sun, J. ClawGuard: A Runtime Security Framework for Tool-Augmented LLM Agents Against Indirect Prompt Injection. *arXiv* **2026**, arXiv:2604.11790. [CrossRef]
17. Li, E.; Mallick, T.; Rose, E.; Robertson, W.; Oprea, A.; Nita-Rotaru, C. ACE: A Security Architecture for LLM-Integrated App Systems. In Proceedings of the Network and Distributed System Security (NDSS) Symposium 2026, San Diego, CA, USA, 23–27 February 2026. [CrossRef]
18. Jia, F.; Wu, T.; Qin, X.; Squicciarini, A. The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Vienna, Austria, 27 July–1 August 2025; pp. 29680–29697. [CrossRef]
19. Denning, D.E. A lattice model of secure information flow. *Commun. ACM* **1976**, *19*, 236–243. [CrossRef]
20. Sabelfeld, A.; Myers, A.C. Language-based information-flow security. *IEEE J. Sel. Areas Commun.* **2003**, *2*, 5–19. [CrossRef]
21. Siddiqui, S.A.; Gaonkar, R.; Köpf, B.; Krueger, D.; Paverd, A.; Salem, A.; Tople, S.; Wutschitz, L.; Xia, M.; Zanella-Béguelin, S. Permissive information-flow analysis for large language models. *arXiv* **2024**, arXiv:2410.03055. [CrossRef]
22. Costa, M.; Köpf, B.; Kolluri, A.; Paverd, A.; Russinovich, M.; Salem, A.; Tople, S.; Wutschitz, L.; Zanella-Béguelin, S. Securing AI agents with information-flow control. *arXiv* **2025**, arXiv:2505.23643. [CrossRef]
23. Crosby, S.A.; Wallach, D.S. Efficient data structures for tamper-evident logging. In Proceedings of the USENIX Security Symposium, Montreal, QC, Canada, 10–14 August 2009; pp. 317–334. Available online: https://www.usenix.org/legacy/event/sec09/tech/full_papers/crosby.pdf (accessed on 26 May 2026).
24. Schneier, B.; Kelsey, J. Secure audit logs to support computer forensics. *ACM Trans. Inf. Syst. Secur.* **1999**, *2*, 159–176. [CrossRef]
25. Soriano-Salvador, E.; Guardiola-Múzquiz, G. SealFS: Storage-based tamper-evident logging. *Comput. Secur.* **2021**, *108*, 102325. [CrossRef]
26. Cosentino, C.; Squillace, S.; Marozzo, F. An Auditable LLM-RAG Architecture for Financial Document Intelligence and Decision Support. *Future Internet* **2026**, *18*, 284. [CrossRef]
27. Ojewale, V.; Steed, R.; Vecchione, B.; Birhane, A.; Raji, I.D. Towards AI Accountability Infrastructure: Gaps and Opportunities in AI Audit Tooling. In Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, Yokohama, Japan, 26 April–1 May 2025. [CrossRef]
28. Saltzer, J.H.; Schroeder, M.D. The protection of information in computer systems. *Proc. IEEE* **1975**, *63*, 1278–1308. [CrossRef]
29. Biba, K.J. *Integrity Considerations for Secure Computer Systems*; No. MTR3153REV1; MITRE Corporation: McLean, VA, USA, 1977.
30. Bell, D.E.; LaPadula, L.J. *Secure Computer Systems: Mathematical Foundations*; No. MTR2547VOL1; MITRE Corporation: McLean, VA, USA, 1973.

31. Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In Proceedings of the International Conference on Learning Representations 2024 (ICLR 2024), Vienna, Austria, 7–11 May 2024. Available online: <https://openreview.net/forum?id=dHng2O0Jjr> (accessed on 26 May 2026).
32. Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; Li, Y. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), Singapore, 6–10 December 2023; pp. 3102–3116. [\[CrossRef\]](#)
33. Yao, S.; Shinn, N.; Razavi, P.; Narasimhan, K. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In Proceedings of the 13th International Conference on Learning Representations (ICLR 2025), Singapore, 24–28 April 2025.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.